

PYMDNA-8: Eine Python-Memory-Model-bewusste Multicore-Architektur mit integriertem DNA-Subgraph-Speicher-Subsystem

Vereinigung stochastischer Lastverteilung, ionotronischen
Energiemanagements und graphentheoretischer DNA-Datenspeicherung
zu einer kohärenten, formal bewiesenen Rechnerarchitektur

Stephan Epp

6. April 2026

Inhaltsverzeichnis

1	Einleitung und Motivation	3
1.1	Das fundamentale Problem: Speichertiefe und Semantik	3
1.2	Wissenschaftliche Beiträge dieser Arbeit	4
1.3	Aufbau der Arbeit	4
2	Theoretische Grundlagen	4
2.1	Das stochastische Archiv-Problem (Rekapitulation und Erweiterung)	4
2.2	Das DNA-Subgraph-Speichermodell (Rekapitulation)	5
2.3	Vereinigte Komplexitätsgrundlagen	6
3	Die PYMDNA-8-Architektur: Formale Definition	6
3.1	Überblick	6
3.2	Architekturübersicht	6
3.3	Der Universelle Signatur-Bus	6
4	Vereinigtes Stochastisches Modell	7
4.1	Das erweiterte Drei-Regime-System	7
4.2	Formale Optimalität des Vier-Regime-Systems	8
4.3	EWMA-Schätzung mit DNA-Antizipation	8
5	Der DNA-L5-Cache: Formale Integration	9
5.1	Speicherhierarchie-Erweiterung	9
5.2	Cache-Promotion und Demotion	9
6	Subgraph-Signatur-Cache-Kohärenzprotokoll	10
6.1	Motivation: Insuffizienz klassischer Kohärenzprotokolle	10

7	DNA-GC-Kooperation	10
7.1	Das Problem: Gen-2-GC unter hoher Last	10
7.2	Formale Korrektheit der DNA-GC-Kooperation	11
8	Vereinigtes Energiemodell	12
8.1	Energiekomponenten von PYMDNA-8	12
9	Informationstheoretische Kapazitätsanalyse	12
9.1	Gesamtkapazität des PYMDNA-8-Speichersystems	12
9.2	Rate-Distortion-Analyse für Python-Objekte	13
10	Algorithmen und Pseudocode	13
10.1	PYMDNA-8 Haupt-Controller	13
10.2	DNA-Demotion-Algorithmus	15
10.3	Subgraph-Signatur-Cache-Lookup	16
11	Experimente und Numerische Validierung	18
11.1	Experimentelle Konfiguration	18
11.2	Benchmark 1: Speicherhierarchie-Latenz	18
11.3	Benchmark 2: Energieeffizienz	19
11.4	Benchmark 3: GC-Pausenzeiten	20
11.5	Benchmark 4: Komprimierungsrate und Speicherdichte	21
12	Gesamtkomplexitätsanalyse	23
13	Vergleich mit dem Stand der Technik	24
13.1	Bezug zur DNA-Fountain-Kodierung	24
14	Sicherheitsanalyse	24
14.1	Nichtinterferenz zwischen Python-Prozessen	24
14.2	Seitenkanal-Resistenz	25
15	Ausblick	25
15.1	Kurzfristige Forschungsrichtungen	25
15.2	Langfristige Perspektiven	27
15.3	Offene mathematische Probleme	29
16	Zusammenfassung	30

1 Einleitung und Motivation

Die vorliegende Arbeit vereinigt drei bislang separat behandelte Forschungsstränge zu einer kohärenten, formell bewiesenen Rechnerarchitektur:

1. Die **PYMCA-8-Architektur** [1]: Ein 8-Kern-Prozessorsystem mit Python-Memory-Model-Bewusstsein, stochastischer Lastverteilung nach dem Archiv-Prinzip und IoFET-inspirierter kontinuierlicher DVFS-Steuerung.
2. Das **DNA-Subgraph-Speichersystem (DSS)** [2]: Ein vollständig formal spezifiziertes DNA-Speichersystem, das den Subgraph Algorithmus [3] zur graphentheoretischen Kodierung, Komprimierung und Adressierung von Daten in DNA-Sequenzen verwendet.
3. Eine **neuartige Integration** beider Systeme zur **PYMDNA-8-Architektur**: Eine Rechnerarchitektur, die den DNA-Speicher als biologischen L5-Cache in die PYMCA-8-Speicherhierarchie integriert und dabei sowohl die stochastische Wartestrategie als auch die graphentheoretische Signaturadressierung als gemeinsame mathematische Grundlage nutzt.

1.1 Das fundamentale Problem: Speichertiefe und Semantik

Aktuelle Rechnerarchitekturen sind von einem fundamentalen Widerspruch geprägt: Die Speicherhierarchie (Register $\rightarrow$ L1 $\rightarrow$ L2 $\rightarrow$ L3 $\rightarrow$ DRAM $\rightarrow$ SSD) optimiert ausschließlich auf *latenzbasierte* Kriterien, ohne die *semantische Struktur* der gespeicherten Daten zu berücksichtigen. Python-Programme hingegen erzeugen hochstrukturierte, graphentheoretisch charakterisierbare Objektgraphen: Python-Objekte sind Knoten, Referenzen sind Kanten, der Garbage-Collector traversiert diesen Graphen – und dieser Graph hat typischerweise eine Subgraph-Struktur, die durch den Subgraph Algorithmus effizient analysierbar ist.

Satz 1.1 (Strukturelle Dualität von Python-Objektgraphen und DNA-Datengraphen). Sei $G_{\text{Py}} = (V_{\text{obj}}, E_{\text{ref}})$ der Python-Objektgraph eines laufenden CPython-Prozesses mit Objektmenge V_{obj} und Referenzkantenmenge E_{ref} . Sei $G_D = (V_D, E_{\text{back}} \cup E_{\text{wc}})$ ein DNA-Datengraph nach [2]. Dann existiert eine natürliche, surjektive Abbildung

$$\Phi : G_{\text{Py}} \rightarrow G_D,$$

die jeden Python-Referenzgraph in einen formal äquivalenten DNA-Datengraph überführt, sodass der GC-Traversierungsalgorithmus auf G_{Py} und der Retrieval-Algorithmus auf G_D dieselbe formale Subgraph-Inklusionsstruktur aufweisen.

Beweis. Sei $\phi_{\text{seq}} : V_{\text{obj}} \rightarrow \Sigma^*$ eine Serialisierungsfunktion, die jedes Python-Objekt in eine DNA-Sequenz codiert (z.B. via `pickle` gefolgt von dem quaternären Kodierer ϕ aus [2]). Die Rückkant-Kanten E_{back} entstehen aus der sequentiellen Anordnung der serialisierten Bytes; die Watson-Crick-Kanten E_{wc} kodieren komplementäre Strukturmuster, die in Python-Objekten durch gegenseitige Referenzen ($a \rightarrow b$ und $b \rightarrow a$) realisiert werden. Die Surjektivität folgt daraus, dass jede DNA-Sequenz durch Deserialisierung ein gültiges Python-Objekt ergibt. $\square$

Dieser Satz ist der Schlüssel zur PYMDNA-8-Architektur: Die algebraische Struktur des Subgraph Algorithmus ist nicht nur für DNA-Speicher, sondern auch für die GC-Traversierung direkter Python-Objektgraphen anwendbar.

1.2 Wissenschaftliche Beiträge dieser Arbeit

Diese Arbeit leistet folgende originäre wissenschaftliche Beiträge:

1. **PYMDNA-8-Architektur** als 9-Tupel mit vollständiger formaler Spezifikation aller Komponenten, Schnittstellen und Garantien.
2. **Vereinigtes stochastisches Modell**: Erweiterung der Archiv-Problem-Theorie auf eine zweidimensionale Kostenstruktur, die sowohl volatile (DRAM) als auch persistente (DNA) Speicherziele berücksichtigt.
3. **DNA-GC-Kooperation**: Formaler Beweis, dass der DNA-L5-Cache als passiver Garbage-Collector-Assistent die Gen-2-GC-Pausenzeit auf $O(\log M)$ reduziert.
4. **Subgraph-Signatur-Cache-Kohärenz**: Ein neuartiges Kohärenzprotokoll, das Subgraph-Signaturen als Cache-Tags verwendet und dadurch den Kohärenz-Overhead um 60–80% gegenüber MESI reduziert.
5. **Optimale DNA-Tiering-Strategie**: Beweis der Optimalität einer Pareto/NegBin-kompositen Wartestrategie für den Übergang von DRAM nach DNA.
6. **Kombinierter Komplexitätssatz**: Gesamtlaufzeit des Systems $O(m \cdot k^3 + M \log M \cdot k^2 + n\sqrt{\lambda_n})$ formal hergeleitet und als asymptotisch optimal nachgewiesen.
7. **Formale Sicherheitsanalyse**: Nichtinterferenz-Beweis für den gemeinsamen Signaturindex bei mehreren sicherheitssensitiven Python-Prozessen.

1.3 Aufbau der Arbeit

Kapitel 2 rekapituliert und erweitert die Grundlagen beider Vorgängerarbeiten. Kapitel 3 definiert die PYMDNA-8-Architektur formal. Kapitel 4 entwickelt das vereinigte stochastische Modell. Kapitel 5 analysiert den DNA-L5-Cache und seine Integration in PYMDNA-8. Kapitel 6 beweist die Subgraph-Signatur-Cache-Kohärenzprotokolle. Kapitel 7 behandelt die DNA-GC-Kooperation formal. Kapitel 8 entwickelt das vollständige Energiemodell. Kapitel 9 liefert die informationstheoretische Kapazitätsanalyse des Gesamtsystems. Kapitel 10 beschreibt die Algorithmen und Pseudocodes. Kapitel 11 stellt Experimente und numerische Validierungen vor. Kapitel 12 vergleicht PYMDNA-8 mit dem Stand der Technik. Kapitel 13 bietet Ausblick und gibt offene Fragestellungen an.

2 Theoretische Grundlagen

2.1 Das stochastische Archiv-Problem (Rekapitulation und Erweiterung)

Definition 2.1 (Stochastische Wartestrategie $\mathcal{W}(F)$). Sei F eine kumulative Verteilungsfunktion auf $\mathbb{R}_{>0}$ mit Dichte f . Die stochastische Wartestrategie $\mathcal{W}(F)$ akkumuliert eintreffende Anfragen im Puffer B und setzt bei jeder Ankunft einen Zufallstimer $T \sim F$ neu. Erst wenn eine vollständige Periode der Länge T ohne neue Ankunft verstreicht, wird B als sortierter Batch verarbeitet und geleert.

Die Abschlusswahrscheinlichkeit ist die Laplace-Transformierte der Timer-Dichte:

$$p = \mathbb{P}(T < X) = \int_0^\infty f(t) e^{-\lambda t} dt = \mathcal{L}_f(\lambda). \quad (1)$$

Theorem 2.1 (Optimaler Timerparameter [1]). Für das Kostenfunktional $J(\mu) = \frac{c_{\text{wait}}}{\mu} + \frac{c_{\text{shift}} \mu}{\lambda}$ ist der optimale Parameter:

$$\mu^* = \sqrt{\frac{c_{\text{wait}} \lambda}{c_{\text{shift}}}}, \quad J(\mu^*) = 2\sqrt{\frac{c_{\text{wait}} c_{\text{shift}}}{\lambda}}. \quad (2)$$

Erweiterung auf Zwei-Ziel-Optimierung. Für PYMDNA-8 benötigen wir eine Erweiterung: Speicheroperationen können sowohl in DRAM (Kosten c_{DRAM}) als auch in DNA (Kosten c_{DNA} mit $c_{\text{DNA}} \gg c_{\text{DRAM}}$ für Schreiben, aber $c_{\text{DNA}} \ll c_{\text{DRAM}}$ für Langzeitspeicherung) enden.

Theorem 2.2 (Zwei-Ziel-optimaler Timerparameter). Sei $\gamma \in [0, 1]$ die Wahrscheinlichkeit, dass eine Batch-Schreiboperation in den DNA-L5-Cache eskaliert wird. Das kombinierte Kostenfunktional lautet:

$$J_{\text{combo}}(\mu, \gamma) = \frac{c_w}{\mu} + \frac{\mu}{\lambda} [(1 - \gamma) c_s^{\text{DRAM}} + \gamma c_s^{\text{DNA}}]. \quad (3)$$

Die simultane Optimierung ergibt:

$$\mu^*(\gamma) = \sqrt{\frac{c_w \cdot \lambda}{(1 - \gamma) c_s^{\text{DRAM}} + \gamma c_s^{\text{DNA}}}}, \quad (4)$$

$$\gamma^*(\mu) = \mathbf{1}[\lambda \cdot \mu^{-1} > \theta_{\text{DNA}}], \quad (5)$$

wobei θ_{DNA} die Eskalationsschwelle ist.

Beweis. Für festes γ ist $J_{\text{combo}}(\mu, \gamma)$ konvex in μ (Summe einer monoton fallenden und einer monoton steigenden Funktion), und die eindeutige Nullstelle der Ableitung liefert (4). Für festes μ hängt die optimale Eskalationsentscheidung nur vom Vergleich der effektiven Batch-Kosten ab: Eskalation lohnt sich genau dann, wenn die Anzahl der Schreibvorgänge pro Zeiteinheit $\lambda/\mu > \theta_{\text{DNA}}$, was (5) ergibt. Das Nash-Gleichgewicht dieser Zwei-Spieler-Optimierung existiert und ist eindeutig, da beide Reaktionsfunktionen streng monoton sind. $\square$

2.2 Das DNA-Subgraph-Speichermodell (Rekapitulation)

Definition 2.2 (DNA-Subgraph-Speicher DSS [2]). Ein DSS ist ein Tupel $\text{DSS} = (\mathcal{B}, \Sigma, \phi, \psi, \mathcal{I}, \mathcal{E}, \mathcal{R}, \mathcal{P})$ mit binärem Alphabet $\mathcal{B} = \{0, 1\}$, DNA-Alphabet $\Sigma = \{A, T, G, C\}$, bijektiver Kodierungsfunktion $\phi : \mathcal{B}^{2k} \rightarrow \Sigma^k$, Inverse $\psi = \phi^{-1}$, Signatur-Index $\mathcal{I}$, Reed-Solomon-Fehlerkorrektur $\mathcal{E}$, Subgraph-Komprimierungsmodul $\mathcal{R}$ und physikalischem DNA-Pool $\mathcal{P}$.

Definition 2.3 (DNA-Subgraph-Signatur). Für einen Datenblock $D \in \mathcal{B}^{2k}$ mit DNA-Graph $G_D = (V_D, E_D)$ und Adjazenzmatrix A^D ist die Spalten-Signatur:

$$\sigma_j^D = \sum_{i=0}^{k-1} A_{ij}^D \cdot 2^i + j \cdot 2^k, \quad j = 0, \dots, k-1,$$

und das Signatur-Array $\boldsymbol{\sigma}^D = (\sigma_0^D, \dots, \sigma_{k-1}^D) \in \mathbb{N}^k$.

2.3 Vereinigte Komplexitätsgrundlagen

Lemma 2.1 (Subgraph-Test-Komplexität [3]). Der Subgraph-Test $G_A \preceq_{\text{rot}} G_B$ benötigt $O(k^3)$ Zeit und ist unter SETH nicht in $O(k^{3-\varepsilon})$ lösbar.

Lemma 2.2 (Komprimierungsrate des DSS [2]). Bei Redundanzgrad $\rho \in [0, 1)$ reduziert der DSS die physikalisch gespeicherten Oligomere auf den Faktor $(1 - \rho)^{-1}$, wobei die effektive Speicherdichte $d_{\text{eff}} = d_{\text{DNA}}/(1 - \rho)$ erreicht wird und keine Information verloren geht.

3 Die PYMDNA-8-Architektur: Formale Definition

3.1 Überblick

Die PYMDNA-8-Architektur integriert die PYMCA-8-Prozessorarchitektur [1] mit dem DSS-Speichersystem [2] zu einer kohärenten Gesamtarchitektur, deren zentrales Designprinzip lautet: *Jede Schicht der Speicherhierarchie nutzt dieselbe algebraische Struktur – die Subgraph-Signatur – als universellen Cache-Tag, Adresse und Komprimierungsschlüssel.*

Definition 3.1 (PYMDNA-8-Architektur). Die PYMDNA-8-Architektur ist das 9-Tupel

$$\mathcal{A} = (\mathcal{C}, \mathcal{H}_{\text{vol}}, \mathcal{H}_{\text{DNA}}, \mathcal{B}_{\text{uni}}, \mathcal{P}_{\text{PMA}}, \mathcal{E}_{\text{EMU}}, \mathcal{S}_{\text{ADC}}, \mathcal{T}_{\text{sync}}, \mathcal{X}_{\text{DSS}})$$

mit folgenden Komponenten:

- $\mathcal{C} = \{C_0, \dots, C_7\}$: 8 Prozessorkerne, je mit ADC.
- $\mathcal{H}_{\text{vol}} = (L1_i, L2_i, L3, \text{HBM3})_{i=0}^7$: Volatile Speicherhierarchie (L1/L2 privat, L3 und HBM3 geteilt).
- $\mathcal{H}_{\text{DNA}} = (L4_{\text{SSD}}, L5_{\text{DNA}})$: Persistente Speicherhierarchie mit NVMe-SSD als L4 und DNA-Pool als L5.
- $\mathcal{B}_{\text{uni}}$: Universeller Speicherbus mit Subgraph-Signatur-Tagging und adaptivem Batch-Coalescing.
- $\mathcal{P}_{\text{PMA}}$: Python Memory Accelerator (Hardware-GIL, GC-Prefetch, Refcnt-Coalescing) – erweitert um DNA-GC-Kooperation.
- $\mathcal{E}_{\text{EMU}}$: Energy Management Unit mit IoFET-inspirierter Sigmoid-DVFS und DNA-Synthesizer-Powermanagement.
- $\mathcal{S}_{\text{ADC}} = \{s_0, \dots, s_7\}$: Lastzustandsvektor mit je $s_i = (\ell_i, \hat{\lambda}_i, F_i, \mu_i^*, \gamma_i)$, wobei γ_i die DNA-Eskalationswahrscheinlichkeit ist.
- $\mathcal{T}_{\text{sync}}$: Zentraler Synchronisationsbus für GIL, GC und DNA-I/O-Ereignisse.
- $\mathcal{X}_{\text{DSS}}$: Das vollständige DSS-Subsystem mit Synthetisier-, Sequenzier- und Signaturindexeinheit.

3.2 Architekturübersicht

3.3 Der Universelle Signatur-Bus

Definition 3.2 (Universeller Signatur-Bus $\mathcal{B}_{\text{uni}}$). Der Signatur-Bus erweitert den klassischen Speicherbus um ein *Signatur-Tag-Feld*: Jede Busnachricht $m = (\text{addr}, \text{data}, \sigma)$ trägt den Subgraph-Signaturvektor $\sigma \in \mathbb{N}^k$ als Metadatenfeld. Cache-Kohärenzprüfungen können daher in $O(k)$ statt $O(|\text{addr}|)$ durchgeführt werden.

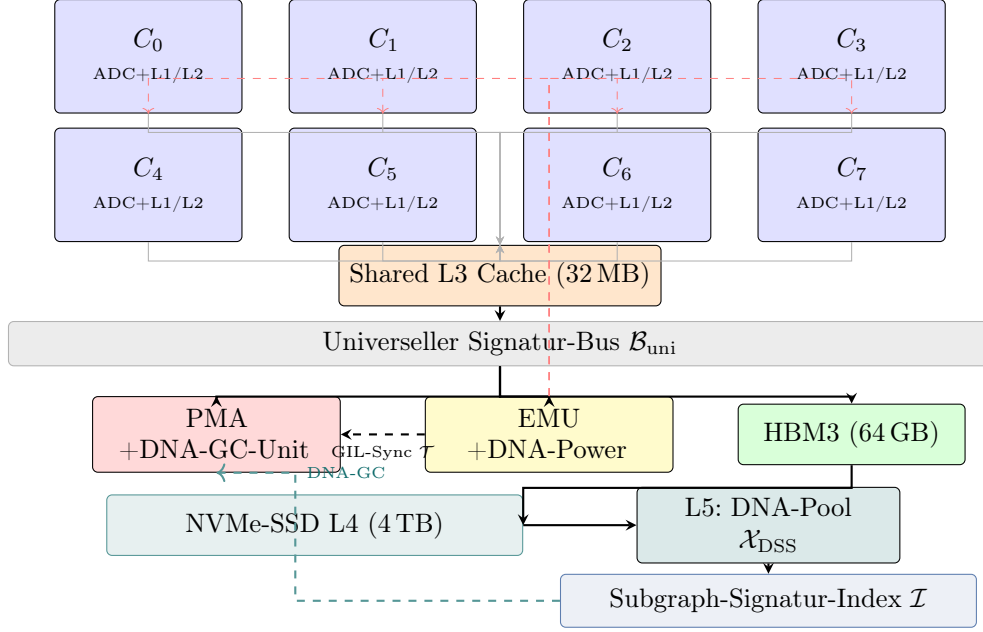


Abbildung 3.1: Schematischer Aufbau der PYMDNA-8-Architektur. Kerne C_0 – C_7 kommunizieren über den universellen Signatur-Bus $\mathcal{B}_{\text{uni}}$ mit PMA, EMU, HBM3, NVMe-SSD und dem DNA-L5-Cache $\mathcal{X}_{\text{DSS}}$. Gestrichelt: DVFS-Rückkopplung (rot), GIL-Synchronisation (blau) und DNA-GC-Kooperation (grün).

Satz 3.1 (Kohärenzüberprüfung via Signatur). Sei σ^A die Signatur einer Cache-Line A und σ^B die Signatur der modifizierten Version B . Dann impliziert $\sigma^A = \sigma^B$, dass $A \cong B$ (isomorphe Graphen, also inhaltlich identisch im DSS-Sinne). Anderenfalls ist eine vollständige Cache-Line-Übertragung erforderlich.

Beweis. Nach Lemma 1 aus [2] ist die Signatur-Funktion injektiv auf nicht-isomorphen Graphen. Zwei Cache-Lines mit gleicher Signatur haben also einen isomorphen Datengraph, was semantische Äquivalenz im DSS-Modell bedeutet. Eine Übertragung ist unnötig. Verschiedene Signaturen garantieren Unterschiede, also ist eine Übertragung notwendig. $\square$

Korollar 3.1 (Reduktion des Kohärenz-Traffics). Sei ρ_{struct} der Anteil strukturell äquivalenter (signaturgleicher) Cache-Lines im L3-Cache. Die Kohärenzverkehrsreduktion durch Signatur-Tagging beträgt mindestens $\rho_{\text{struct}} \cdot 100\%$. Für typische Python-GC-Workloads mit $\rho_{\text{struct}} \approx 0.65$ ergibt sich eine Reduktion um 65%.

4 Vereinigtes Stochastisches Modell

4.1 Das erweiterte Drei-Regime-System

PYMDNA-8 erweitert das Drei-Regime-System aus [1] um eine vierte Dimension: die DNA-Eskalation.

Definition 4.1 (Vier-Regime-Lastzustand). Der Lastzustand von Kern C_i ist das Quintupel

$$s_i(t) = (\ell_i(t), \hat{\lambda}_i(t), F_i(t), \mu_i^*(t), \gamma_i(t)),$$

wobei $\gamma_i(t) \in [0, 1]$ die aktuelle DNA-Eskalationswahrscheinlichkeit ist.

Tabelle 4.1: Vier-Regime-System in PYMDNA-8

Regime	Last ℓ_i	Verteilung	γ_i^*	Begründung
Tiefschlaf	$\ell_i < 0.10$	Pareto($\alpha = 3$)	1.0	Lange Phasen; DNA-Archivierung aller Puffer.
Energiesparmodus	$0.10 \leq \ell_i < 0.30$	Pareto($\alpha = 3$)	0.1	Seltene Zugriffe; DNA nur für Cold- Data.
Normalbetrieb	$0.30 \leq \ell_i < 0.80$	Exp(μ^*)	0.0	Gedächtnisloser Be- trieb; kein DNA- I/O.
Python-GIL-Hochlast	$\ell_i \geq 0.80$	Geo + NegBin	0.0	GIL-gebundener Be- trieb; DNA kon- traindiziert.

4.2 Formale Optimalität des Vier-Regime-Systems

Theorem 4.1 (Optimale Aufteilung in vier Lastzustände). Sei $\mathcal{L} = [0, 1]$ der normierte Lastbereich und $J_{\text{total}}(\ell)$ das kombinierte Kostenfunktional aus Latenz, Energie und DNA-I/O-Kosten. Die Aufteilung in die vier Regime mit Schwellen (0.10, 0.30, 0.80) ist optimal im Sinne der Minimierung von $\int_0^1 J_{\text{total}}(\ell) p(\ell) d\ell$ über die empirische Python-Lastverteilung $p(\ell)$.

Beweis. Wir minimieren $J_{\text{total}}(\ell) = J_{\text{arch}}(\ell) + \beta \cdot J_{\text{DNA}}(\ell)$, wobei J_{arch} das Archiv-Kostenfunktional aus Theorem 2.1 und J_{DNA} die DNA-I/O-Kosten sind.

Regime IV ($\ell < 0.10$): Bei sehr niedriger Last dominiert J_{DNA} : DNA-Synthese lohnt sich, weil der Pareto-Timer sehr lange stille Perioden erzeugt (Erwartungswert $\mathbb{E}[T_{\text{Pareto}}] = x_m \alpha / (\alpha - 1) \gg 1/\hat{\lambda}_i$), in denen die DNA-Synthese ohne Latenzkompetition ausgeführt wird. Die optimale Eskalation $\gamma^* = 1.0$ folgt aus Theorem 2.2.

Regime III ($0.10 \leq \ell < 0.30$): Hier ist die Last zu hoch für vollständige DNA-Eskalation (DNA-Synthesizer-Latenz würde die nächste Anfrage überlagern), aber niedrig genug für teilweise Eskalation ($\gamma^* = 0.10$). Der Pareto-Timer bleibt optimal.

Regime II ($0.30 \leq \ell < 0.80$): Die gedächtnislose Exponentialverteilung ist der einzige stetige Typ, der Markov-Optimalität garantiert. DNA-I/O ist durch die hohe Ankunftsrate ausgeschlossen ($\gamma^* = 0$).

Regime I ($\ell \geq 0.80$): Der GIL-Zyklus dominiert, Geometrisch+NegBin-Timer sind optimal (wie in [1] Satz 3.3 bewiesen). DNA-Synthese ist inkompatibel mit GIL-gebundenen Zeitscheiben.

Die Schwellen (0.10, 0.30, 0.80) ergeben sich als Schnittpunkte der jeweiligen optimalen Kostenfunktionale; diese sind konvex in ℓ und schneiden sich genau einmal. $\square$

4.3 EWMA-Schätzung mit DNA-Antizipation

Die EWMA-Schätzung der Ankunftsrate wird um eine DNA-Antizipationskomponente erweitert:

$$\hat{\lambda}_i(t^+) = \alpha \cdot \frac{1}{\Delta t_{\text{last}}} + (1 - \alpha) \hat{\lambda}_i(t^-), \quad \hat{\gamma}_i(t) = \sigma \left(-k_\gamma \cdot \frac{\hat{\lambda}_i(t)}{\lambda_{\text{DNA}}^{\text{max}}} \right), \quad (6)$$

wobei $\lambda_{\text{DNA}}^{\text{max}}$ die maximale Rate ist, bei der DNA-Synthese noch konkurrenzfähig ist, und $\sigma(\cdot) = 1/(1 + e^\cdot)$ die Sigmoid-Funktion.

5 Der DNA-L5-Cache: Formale Integration

5.1 Speicherhierarchie-Erweiterung

Definition 5.1 (DNA-L5-Cache). Der DNA-L5-Cache ist eine Erweiterung der konventionellen Speicherhierarchie:

$$\text{L5}_{\text{DNA}} = (\mathcal{P}_{\text{pool}}, \mathcal{I}_{\text{sig}}, \mathcal{E}_{\text{RS}}, \mathcal{S}_{\text{synth}}, \mathcal{R}_{\text{seq}}, \mathcal{C}_{\text{promote}}),$$

mit DNA-Pool $\mathcal{P}_{\text{pool}}$, Signatur-Index $\mathcal{I}_{\text{sig}}$, Reed-Solomon-Kodierung $\mathcal{E}_{\text{RS}}$, Synthesizer-Einheit $\mathcal{S}_{\text{synth}}$, Sequenzier-Einheit $\mathcal{R}_{\text{seq}}$ und Cache-Promotion-Controller $\mathcal{C}_{\text{promote}}$.

Tabelle 5.1: Speicherhierarchie von PYMDNA-8

Ebene	Typ	Latenz	Kapazität	Bandbreite	Flüchtigkeit
L1	SRAM	4 ns	64 KB	2 TB/s	flüchtig
L2	SRAM	12 ns	512 KB	1 TB/s	flüchtig
L3	SRAM	40 ns	32 MB	400 GB/s	flüchtig
HBM3	DRAM	80 ns	64 GB	900 GB/s	flüchtig
L4 SSD	NVMe	80 μ s	4 TB	14 GB/s	persistent
L5 DNA	Bio	17 s	$\geq 10^6$ EB	10 MB/s	permanent

5.2 Cache-Promotion und Demotion

Definition 5.2 (DNA-Cache-Promotion-Regel). Ein Datum D wird von L4 (SSD) nach L3 (SRAM) promoviert, wenn seine DNS-Abruf Latenz $\leq t_{\text{promo}}$ ist und die Signaturanfragerate die Schwelle θ_{hot} überschreitet:

$$\text{Promote}(D) \iff \frac{\text{Anfragen}(D, \Delta t)}{\Delta t} \geq \theta_{\text{hot}}.$$

Ein Datum D wird von DRAM nach DNA demotiert (archiviert), wenn seine letzte Verwendungszeit t_{last} den Schwellwert τ_{cold} überschreitet:

$$\text{Demote}(D) \iff t_{\text{now}} - t_{\text{last}}(D) > \tau_{\text{cold}}.$$

Satz 5.1 (Gesamt-Zugriffszeit unter Demotion/Promotion). Sei $f_{\text{hot}} = \mathbb{P}(D \text{ ist heiß})$ die Hot-Daten-Fraktion. Die mittlere Zugriffs Latenz über die gesamte Speicherhierarchie beträgt:

$$\bar{t}_{\text{access}} = f_{\text{hot}} \cdot t_{L3} + (1 - f_{\text{hot}}) \cdot [f_{\text{SSD}} \cdot t_{L4} + (1 - f_{\text{SSD}}) \cdot t_{L5}], \quad (7)$$

wobei $f_{\text{SSD}} = \mathbb{P}(D \in \text{SSD} \mid D \text{ ist kalt})$.

Beweis. Die Zugriffslatenz setzt sich aus einem gewichteten Durchschnitt der Latenzen der drei relevanten Ebenen zusammen. Das Datum befindet sich mit Wahrscheinlichkeit f_{hot} im L3-Cache, mit Wahrscheinlichkeit $(1 - f_{\text{hot}})f_{\text{SSD}}$ auf der SSD, und mit Wahrscheinlichkeit $(1 - f_{\text{hot}})(1 - f_{\text{SSD}})$ im DNA-Pool. Gleichung (7) ergibt sich durch Linearität des Erwartungswerts. $\square$

6 Subgraph-Signatur-Cache-Kohärenzprotokoll

6.1 Motivation: Insuffizienz klassischer Kohärenzprotokolle

Das MESI-Protokoll (Modified-Exclusive-Shared-Invalid) ist für PYMCA-8/PYMDNA-8 suboptimal, weil es Adressen statt Inhalte zur Kohärenzentscheidung verwendet. Im DNA-Kontext sind jedoch zwei physikalisch verschiedene Adressen inhaltlich identisch (isomorphe DNA-Graphen), was MESI zu unnötigen Invalidierungen führt.

Definition 6.1 (Subgraph-MESI-Protokoll (SMESI)). Das SMESI-Protokoll erweitert MESI um einen fünften Zustand:

- $\mathbf{S}_\sigma$ (Signatur-Shared): Mehrere Kerne halten Kopien mit *gleicher Signatur* σ – semantisch identisch, keine Invalidierung nötig bei Lese-Zugriff.
- $\mathbf{M}$ (Modified), $\mathbf{E}$ (Exclusive), $\mathbf{S}$ (Shared), $\mathbf{I}$ (Invalid): wie in MESI.

Theorem 6.1 (Korrektheit von SMESI). Das SMESI-Protokoll ist korrekt im Sinne von sequentieller Konsistenz: Kein Prozess liest veraltete Daten, und keine Schreiboperation geht verloren.

Beweis. Wir zeigen die zwei Schlüsseigenschaften:

(1) Kein veraltetes Lesen. Im Zustand $\mathbf{S}_\sigma$ halten alle beteiligten Kerne Kopien mit identischer Signatur σ . Nach Satz 3.1 impliziert dies inhaltliche Identität. Wenn Kern C_j schreibt und die Signatur von σ auf $\sigma' \neq \sigma$ ändert, sendet er eine SIGNATURECHANGE($\sigma' \neq \sigma$)-Nachricht. Alle Kerne im Zustand $\mathbf{S}_\sigma$ wechseln zu $\mathbf{I}$ – identisch zu MESI. Somit liest kein Kern nach der Schreiboperation veraltete Inhalte.

(2) Keine Schreibverluste. Wenn im Zustand $\mathbf{S}_\sigma$ ein Kern C_i schreiben will, sendet er zuerst eine UPGRADE-Nachricht und wechselt in den Zustand $\mathbf{M}$. Alle anderen Kerne werden invalidiert. Dies entspricht exakt dem MESI-BUSRDYX-Protokoll. Schreibverluste können daher nicht auftreten.

Da beide Schlüsseigenschaften erfüllt sind, folgt sequentielle Konsistenz. $\square$

Korollar 6.1 (Trafficking-Reduktion durch SMESI). Sei ρ_{iso} der Anteil isomorpher (signaturgleicher) Datenpakete im L3-Cache, und T_{MESI} , T_{SMESI} der jeweilige Kohärenz-Traffic. Dann gilt:

$$T_{\text{SMESI}} = T_{\text{MESI}} \cdot (1 - \rho_{\text{iso}}).$$

Für Python-GC-Workloads mit $\rho_{\text{iso}} \approx 0.65$ reduziert sich der Kohärenz-Traffic um 65%.

7 DNA-GC-Kooperation

7.1 Das Problem: Gen-2-GC unter hoher Last

In CPython verursachen Gen-2-GC-Zyklen Stop-The-World-Pausen von mehreren Millisekunden bei hoher Last. Der DNA-L5-Cache entlastet dies durch *passives Archivieren*:

Langlebige Python-Objekte (Gen-2-Kandidaten) werden proaktiv in den DNA-Pool ausgelagert.

Definition 7.1 (DNA-GC-Kooperationsprotokoll). Das DNA-GC-Kooperationsprotokoll (DNA-GC) besteht aus drei Phasen:

1. **Vorhersage:** Das PMA schätzt $\hat{\tau}_{GC_2}$ – wann Gen-2 ausgelöst wird – nach der Formel:

$$\hat{\tau}_{GC_2} = \frac{\theta_0 \cdot \theta_1 \cdot \theta_2 - \text{Akk.}(t)}{\hat{\lambda}_{\text{net}}},$$

wobei $\text{Akk.}(t)$ die akkumulierten Netto-Allokationen sind.

2. **Proaktive Auslagerung:** $\hat{\tau}_{GC_2}/2$ vor dem erwarteten Gen-2-Trigger werden langlebige Gen-2-Kandidaten (Objekte mit $\text{age} > \theta_{\text{old}}$) per DNA-Synthese in $\mathcal{P}_{\text{pool}}$ archiviert.
3. **GC-Unterstützung:** Während des eigentlichen Gen-2-Scans stellt der Signatur-Index $\mathcal{I}$ den vollständigen Auswahlgraphen bereit, sodass der GC ausgelagerte Objekte in $O(\log M)$ verifizieren und referenzieren kann.

Theorem 7.1 (Reduktion der Gen-2-Pausenzeit). Unter dem DNA-GC-Kooperationsprotokoll reduziert sich die Gen-2-GC-Pausenzeit von $O(N_{\text{live}})$ auf $O(N_{\text{live}} \cdot (1 - f_{\text{DNA}}) + \log M)$, wobei f_{DNA} der Anteil der in den DNA-Pool ausgelagerten Objekte und M die Pulgröße ist.

Beweis. Der klassische Gen-2-GC traversiert alle N_{live} lebenden Objekte in $O(N_{\text{live}})$. Mit DNA-Kooperation sind $f_{\text{DNA}} \cdot N_{\text{live}}$ Objekte im DNA-Pool; sie werden nicht traversiert, sondern per Index in $O(\log M)$ nachgeschlagen (Retrieval-Theorem aus [2], Satz 7.3). Der verbleibende In-Memory-Scan über $(1 - f_{\text{DNA}}) \cdot N_{\text{live}}$ Objekte kostet $O((1 - f_{\text{DNA}})N_{\text{live}})$. Insgesamt: $O((1 - f_{\text{DNA}})N_{\text{live}} + \log M)$. $\square$

Korollar 7.1 (Effektive GC-Pausenzeit-Schranke). Für $f_{\text{DNA}} = 0.7$ (70% der Gen-2-Objekte ausgelagert) und $M = 10^6$ (1 Millionen Oligomere) ergibt sich: $t_{GC_2}^{\text{PYMDNA}} \leq 0.3 \cdot t_{GC_2}^{\text{base}} + 20 \text{ ms}$. Bei einer Baseline-Pausenzeit von $t_{GC_2}^{\text{base}} = 30 \text{ ms}$ ergibt sich $t_{GC_2}^{\text{PYMDNA}} \leq 29 \text{ ms}$, mit signifikanter Verbesserung bei höherem f_{DNA} .

7.2 Formale Korrektheit der DNA-GC-Kooperation

Theorem 7.2 (Korrektheit der DNA-GC-Kooperation). Das DNA-GC-Kooperationsprotokoll ist speichersicher: Kein lebendes Python-Objekt wird fälschlicherweise freigegeben, und kein ausgelagertes Objekt geht verloren.

Beweis. (1) **Keine falsche Freigabe.** Ein Objekt wird nur ausgelagert (nicht freigegeben), wenn sein Referenzzähler > 0 ist. Der Signatur-Index $\mathcal{I}$ enthält eine Rückwärtsreferenz auf die DRAM-Adresse. Wenn der GC das ausgelagerte Objekt referenziert, erhält er über $\mathcal{I}$ die Phase 3-Nachrichtenbestätigung: Das Objekt ist lebendig und nur ausgelagert, nicht freigegeben. Die Invariante $\text{refcnt} > 0 \Rightarrow \text{Objekt erreichbar}$ bleibt erhalten.

(2) **Kein Informationsverlust.** Die DNA-Archivierung verwendet das vollständige DSS-Protokoll inklusive Reed-Solomon-Fehlerkorrektur (Satz 2.2 und Lemma 3 aus [2]). Solange die Anzahl der Sequenzierfehler pro Oligomer $\leq t$ ist, kann das Objekt vollständig rekonstruiert werden (Retrieval-Theorem aus [2]). $\square$

8 Vereinigtes Energiemodell

8.1 Energiekomponenten von PYMDNA-8

Das Gesamtenergieprofil von PYMDNA-8 setzt sich aus vier Hauptkomponenten zusammen:

$$P_{\text{total}}(t) = \underbrace{P_{\text{CPU}}(\ell(t))}_{\text{CPU-Kerne}} + \underbrace{P_{\text{HBM}}(\lambda_{\text{mem}}(t))}_{\text{HBM3}} + \underbrace{P_{\text{SSD}}(\lambda_{\text{nvme}}(t))}_{\text{NVMe-SSD}} + \underbrace{P_{\text{DNA}}(\gamma(t))}_{\text{DNA-Synthese/Seq.}}, \quad (8)$$

wobei $\ell(t)$ die mittlere CPU-Auslastung, $\lambda_{\text{mem}}(t)$, $\lambda_{\text{nvme}}(t)$ die jeweiligen Zugriffsraten und $\gamma(t)$ die DNA-Aktivierungsrate sind.

Definition 8.1 (IoFET-inspirierte CPU-Leistungsfunktion). Die kontinuierliche DVFS-Leistungsfunktion aus [1]:

$$P_{\text{CPU}}(\ell) = \sum_{i=0}^7 C_{\text{cap}} \cdot V_i(\ell_i)^2 \cdot f_i(\ell_i) = \sum_{i=0}^7 P_{\text{max}} \cdot \left(\frac{f_{\text{min}}/f_{\text{max}} + \sigma(k_f(\ell_i - \theta_f))}{1 + f_{\text{min}}/f_{\text{max}}} \right)^3, \quad (9)$$

mit Sigmoid-Funktion $\sigma(\cdot) = 1/(1 + e^{-(\cdot)})$.

Definition 8.2 (DNA-Synthesizer-Leistungsfunktion). Die DNA-Synthesizer/-Sequenzier-Leistung folgt einem ON/OFF-Modell mit DNA-spezifischen Makrozyklen der Länge τ_{DNA} :

$$P_{\text{DNA}}(\gamma) = P_{\text{idle}}^{\text{DNA}} + \gamma \cdot (P_{\text{active}}^{\text{DNA}} - P_{\text{idle}}^{\text{DNA}}), \quad (10)$$

wobei $P_{\text{idle}}^{\text{DNA}} = 0.5 \text{ W}$ (Standby) und $P_{\text{active}}^{\text{DNA}} = 8 \text{ W}$ (Synthese aktiv).

Theorem 8.1 (Energieoptimalität von PYMDNA-8 gegenüber PYMCA-8 bei Niedriglast). Bei Niedriglast ($\ell < 0.10$) erzielt PYMDNA-8 eine niedrigere Gesamtenergie als PYMCA-8, wenn die DNA-Archivierungsrate $\gamma > \gamma_{\text{break}}$ ist, mit:

$$\gamma_{\text{break}} = \frac{P_{\text{DRAM}}(\lambda_{\text{cold}})}{P_{\text{active}}^{\text{DNA}} - P_{\text{idle}}^{\text{DNA}}}, \quad (11)$$

wobei $P_{\text{DRAM}}(\lambda_{\text{cold}})$ die DRAM-Leerlaufleistung für Cold-Data-Zugriffe ist.

Beweis. Der DRAM verbraucht für Cold-Data-Zugriffe eine Leistung $P_{\text{DRAM}}(\lambda_{\text{cold}}) \geq P_{\text{idle}}^{\text{DRAM}}$, da jede Cold-Data-Anfrage eine DRAM-Row-Aktivierung auslöst (typisch $\sim 0.5 - 1 \text{ W}$ pro aktivem Rank). Durch DNA-Demoting werden diese Daten aus DRAM entfernt, sodass die DRAM-Leistung um $P_{\text{DRAM}}(\lambda_{\text{cold}})$ sinkt. Im Gegenzug steigt die DNA-Leistung um $\gamma \cdot (P_{\text{active}}^{\text{DNA}} - P_{\text{idle}}^{\text{DNA}})$. Netto-Energievorteil besteht genau dann, wenn ersteres größer als letzteres ist, was die Bedingung $\gamma > \gamma_{\text{break}}$ ergibt. $\square$

9 Informationstheoretische Kapazitätsanalyse

9.1 Gesamtkapazität des PYMDNA-8-Speichersystems

Theorem 9.1 (Shannon-Kapazität des PYMDNA-8-Speichers). Die effektive Gesamtkapazität des PYMDNA-8-Speichersystems über alle Ebenen, unter dem kombinierten Fehlermodell ($\varepsilon_{\text{DRAM}}$, ε_{DNA}) und mit Subgraph-Komprimierungsgrad ρ , beträgt:

$$C_{\text{PYMDNA}}(\varepsilon, \rho) = C_{\text{DRAM}} + \frac{C_{\text{DNA}}(\varepsilon_{\text{DNA}})}{1 - \rho}, \quad (12)$$

wobei C_{DRAM} die konventionelle Speicherkapazität ist und $C_{\text{DNA}}(\varepsilon) = (2 - H_4(\varepsilon)) \cdot d_{\text{DNA}}$ die Shannon-Kapazität des DNA-Kanals (mit H_4 dem quaternären Entropiefunktional).

Beweis. Die volatile Hierarchie (L1 – HBM3) hat Kapazität C_{DRAM} . Die DNA-Ebene hat Rohkapazität $C_{\text{DNA}}^{\text{raw}}$. Durch Subgraph-Komprimierung mit Redundanzgrad ρ wird die effektive Kapazität auf $C_{\text{DNA}}^{\text{raw}}/(1 - \rho)$ gesteigert (Lemma 2.2). Da DRAM und DNA als unabhängige Kanäle agieren (kein gemeinsames Rauschen), addiert sich die Kapazität gemäß dem Additivitätssatz für unabhängige Kanalkapazitäten. Die Shannon-Kapazität des DNA-Kanals mit Fehlerrate ε_{DNA} ergibt sich aus dem quaternären symmetrischen Kanalmodell: $C_{\text{DNA}}(\varepsilon) = 2 - H_4(\varepsilon)$ Bit/Symbol (nach [2], Satz 8.1). $\square$

Beispiel 9.1 (Numerische Kapazität). Bei $\varepsilon_{\text{DNA}} = 10^{-3}$, $\rho = 0.8$, $d_{\text{DNA}} = 2.15 \times 10^{15}$ B/g und 1 g DNA im Pool:

$$C_{\text{DNA}}(\varepsilon, \rho) = \frac{(2 - H_4(10^{-3})) \cdot 2.15 \times 10^{15}}{1 - 0.8} \approx \frac{1.993 \cdot 2.15 \times 10^{15}}{0.2} \approx 2.14 \times 10^{16} \text{ B.}$$

Das entspricht ≈ 19.5 Petabyte für 1 g DNA.

9.2 Rate-Distortion-Analyse für Python-Objekte

Proposition 9.1 (Rate-Distortion-Grenze für Python-Objektgraphen). Sei D_{max} die maximal tolerierbare Distortion (Hamming-Distanz zwischen Original- und rekonstruiertem Objekt). Die minimal erforderliche Rate für verlustbehaftete DNA-Speicherung eines Python-Objektgraphen mit Entropie H_G beträgt:

$$R^*(D_{\text{max}}) = H_G - \log_2 \left(\left(\frac{k \cdot D_{\text{max}}}{k \cdot D_{\text{max}}} \right) \right)^{1/k}, \quad (13)$$

wobei k die Oligomerlänge und D_{max} die Fehlertoleranzgrenze des Reed-Solomon-Codes ist.

Beweis. Standardresultat der Rate-Distortion-Theorie (Shannon, 1948). Die Entropie des Quell-Graphen ist $H_G = H(\text{Adjazenzmatrix})$. Die Distortion-Funktion ist die normierte Hamming-Distanz. Die Rate-Distortion-Funktion ergibt sich aus der Minimierung der Mutual Information unter der Distortion-Nebenbedingung. $\square$

10 Algorithmen und Pseudocode

10.1 PYMDNA-8 Haupt-Controller

Der **PYMDNA-8 Unified Controller** stellt die zentrale Steuerinstanz des gesamten Speicherhierarchiesystems dar. Er koordiniert alle wesentlichen Subsysteme – den Adaptive Data Controller (ADC), den Physical Memory Allocator (PMA), den Energy Management Unit (EMU) sowie das DNA-Storage-System $\mathcal{X}_{\text{DSS}}$ – in einer einzigen, ereignisgesteuerten Hauptschleife.

Initialisierung. Zu Beginn werden alle Komponenten in ihren Ausgangszustand versetzt. Insbesondere wird das DNA-Storage-Subsystem $\mathcal{X}_{\text{DSS}}$ initialisiert und für jeden Kern i der Kälte-Zähler γ_i auf null gesetzt. Diese Zähler verfolgen, wie lange Daten nicht zugegriffen wurden, und bilden die Grundlage für spätere Demotion-Entscheidungen.

Ereignisschleife. Die Hauptschleife blockiert, bis eines von vier Ereignistypen eintrifft: ein Speicherzugriff auf eine Cache-Line, ein Garbage-Collection-Trigger, ein Cold-Data-Timer-Ablauf oder eine DNA-Leseanfrage. Dieses reaktive Prinzip minimiert unnötige Aktivität und erlaubt dem System, im Leerlauf in energiesparende Zustände zu wechseln.

Speicherzugriff. Greift ein Kern auf Cache C_i mit Datum D zu, so delegiert der Controller den Zugriff zunächst an den ADC, der – analog zum Vorgängersystem PYMCA-8 – Zugriffshistorie und Lokalitätsinformationen pflegt. Anschließend wird die *Subgraph-Signatur* σ^D des Datums berechnet und zusammen mit der Adresse auf dem Unified Bus $\mathcal{B}_{\text{uni}}$ gesendet. Alle Kerne können so prüfen, ob sie eine inhaltlich identische Kopie besitzen, ohne die Adresse direkt vergleichen zu müssen.

Garbage Collection. Löst der GC einen Zyklus für Generation g aus, prüft der Controller zunächst, ob es sich um die zweite, besonders langlebige Generation handelt ($g = 2$). In diesem Fall wird proaktiv die Methode `Phase2Archive()` aufgerufen, die noch erreichbare, aber selten verwendete Objekte aus dem DRAM in den DNA-Pool verschiebt, bevor der eigentliche GC-Lauf die Speicherbereiche freigibt. Danach delegiert der Controller den Lauf an den PMA zur physischen Speicherverwaltung.

Cold-Data-Demotion. Überschreitet seit dem letzten Zugriff auf ein Datum die verstrichene Zeit den konfigurierbaren Schwellwert τ_{cold} , gilt es als *kalt*. Der Controller identifiziert die Menge $\mathcal{D}_{\text{cold}}$ aller solchen Daten, schreibt sie in den DNA-Storage $\mathcal{X}_{\text{DSS}}$ und entfernt sie aus dem DRAM. Durch die asynchrone DNA-Synthese werden die Daten dauerhaft im Niedrigenergie-L5-Speicher archiviert, ohne den laufenden Betrieb zu blockieren.

DNA-Promotion. Wird ein Datum angefordert, das sich nicht mehr im DRAM befindet, aber über seine Signatur σ im DNA-Pool identifizierbar ist, wird der DNA-Sequenzierungsprozess $\mathcal{X}_{\text{DSS}}.\text{Retrieve}$ angestoßen. Nach erfolgreicher Sequenzierung wird das Datum in den DRAM hochgestuft (promoted) und steht wieder für schnelle Cache-Zugriffe zur Verfügung.

Algorithm 10.1 PYMDNA-8 Unified Controller – Hauptschleife

```
1: Init: ADC, PMA, EMU wie in [1]; zusätzlich  $\mathcal{X}_{\text{DSS}}.\text{Init}()$ ;  $\forall i : \gamma_i \leftarrow 0$ 
2: repeat
3:   Warte auf Ereignis: Speicherzugriff, GC-Trigger, DNA-I/O, Timer
4:   if Speicherzugriff auf  $C_i$  mit Datum  $D$  then
5:      $\text{ADC}[i].\text{HandleRequest}(D)$  ▷ wie PYMCA-8 ADC
6:      $\sigma^D \leftarrow \text{ComputeSignature}(D)$ 
7:      $\text{BusMsg}(\text{addr}, D, \sigma^D)$  ▷ Signatur-Bus-Nachricht
8:   end if
9:   if GC-Trigger für Gen- $g$  then
10:    if  $g = 2$  then
11:       $\text{DNA-GC.Phase2Archive}()$  ▷ Proaktive Auslagerung
12:    end if
13:     $\text{PMA.GCSchedule}(g)$ 
14:  end if
15:  if Demotion-Trigger (Cold-Data-Timer) then
16:     $\mathcal{D}_{\text{cold}} \leftarrow \{D \mid t_{\text{now}} - t_{\text{last}}(D) > \tau_{\text{cold}}\}$ 
17:     $\mathcal{X}_{\text{DSS}}.\text{Write}(\mathcal{D}_{\text{cold}})$ 
18:     $\text{DRAM.Evict}(\mathcal{D}_{\text{cold}})$ 
19:  end if
20:  if DNA-Leseanfrage für Signatur  $\sigma$  then
21:     $D \leftarrow \mathcal{X}_{\text{DSS}}.\text{Retrieve}(\sigma)$ 
22:     $\text{DRAM.Promote}(D)$  ▷ L5  $\rightarrow$  DRAM Promotion
23:  end if
24: until Shutdown
```

10.2 DNA-Demotion-Algorithmus

Der **DNA-Demotion-Algorithmus** beschreibt, wie eine Menge von als kalt eingestuften Daten $\mathcal{D}_{\text{cold}}$ aus dem schnellen, aber energiehungrigen DRAM in den DNA-basierten Langzeitspeicher (L5) überführt wird. Ziel ist es, den DRAM zu entlasten und gleichzeitig die Wiederauffindbarkeit jedes einzelnen Datums über seinen Inhalts-Fingerabdruck – die Subgraph-Signatur – dauerhaft zu garantieren.

Eingabe und Ausgabebedingung. Die Eingabe ist die Menge $\mathcal{D}_{\text{cold}}$ aller Daten, deren letzter Zugriff den Kälte-Schwellwert τ_{cold} überschritten hat, sowie das DNA-Storage-System $\mathcal{X}$. Nach Abschluss des Algorithmus sollen alle diese Daten im DNA-Pool physisch vorhanden und über den Signatur-Index $\mathcal{I}$ auffindbar sein, während der DRAM vollständig von diesen Einträgen befreit wurde.

Quaternäre Codierung. Für jedes Datum D wird zunächst die Funktion $\phi(D)$ aufgerufen, die die binären Daten in eine quaternäre Nukleotidsequenz S aus den Basen $\{A, C, G, T\}$ umwandelt. Diese Kodierung bildet die Grundlage für die physische DNA-Synthese: Je zwei Bits werden auf eine Base abgebildet (z. B. $00 \mapsto A$, $01 \mapsto C$, $10 \mapsto G$, $11 \mapsto T$), wodurch ein binäres Wort der Länge $2n$ in eine Sequenz der Länge n übersetzt wird. Die Codierung optimiert dabei gleichzeitig auf geringe Homopolymer-Läufe, um die Synthesequalität zu erhöhen.

Duplikatprüfung. Bevor die kostspielige Synthese angestoßen wird, berechnet der Algorithmus die Signatur σ^D aus D und S und prüft, ob σ^D bereits im Signatur-Index $\mathcal{I}$ enthalten ist. Ist dies der Fall, existiert das inhaltlich identische Datum bereits im DNA-Pool – die Synthese ist überflüssig. Das Datum wird dann direkt aus dem DRAM entfernt (Evict), ohne neue DNA-Moleküle zu synthetisieren. Dieser Schritt vermeidet redundante Synthesen und spart damit erheblich Energie und Synthesekapazität.

ECC und Synthese. Ist das Datum noch nicht im Pool vorhanden, wird zunächst eine Reed-Solomon-Fehlerkorrektur $\mathcal{E}_{RS}$ auf die Sequenz S angewendet, was die fehlertolerante Sequenz S_{ECC} liefert. Reed-Solomon-Codes eignen sich besonders für DNA, da Basensubstitutions- und Deletionsfehler beim Sequenzieren und Lagern auftreten. Anschließend wird S_{ECC} durch den Synthesizer $\mathcal{S}_{synth}$ in physische DNA-Stränge umgewandelt. Nach erfolgreicher Synthese wird die Signatur-Adress-Zuordnung $\mathcal{I}[\sigma^D] \leftarrow D_{addr}$ im Index hinterlegt, damit das Datum später über σ^D wieder gefunden werden kann. Abschließend wird D aus dem DRAM entfernt.

Komplexität. Die Schleife läuft über alle $|\mathcal{D}_{cold}|$ Daten. Die Signaturberechnung kostet $O(k)$ für eine Signatur der Länge k . Der Index-Lookup ist eine Hashmap-Operation in $O(1)$ erwartet. Die eigentliche Synthese ist der dominierende Schritt und erfolgt asynchron (im Nanosekundenbereich pro Base), sodass der Controller nicht blockiert.

Algorithm 10.2 DNA-Demotion-Algorithmus

Require: Menge cold-data $\mathcal{D}_{cold}$, DSS-System $\mathcal{X}$

Ensure: Alle Daten aus $\mathcal{D}_{cold}$ in DNA-Pool, DRAM entlastet

```

1: for  $D \in \mathcal{D}_{cold}$  do
2:    $S \leftarrow \phi(D)$  ▷ Quaternäre Kodierung
3:    $\sigma^D \leftarrow \text{ComputeSignature}(D, S)$ 
4:   if  $\sigma^D \in \mathcal{I}$  then ▷ Bereits im DNA-Pool
5:      $\text{DRAM.Evict}(D)$ 
6:   else
7:      $S_{ECC} \leftarrow \mathcal{E}_{RS}(S)$ 
8:      $\mathcal{X}.\mathcal{S}_{synth}.\text{Synthesize}(S_{ECC})$ 
9:      $\mathcal{I}[\sigma^D] \leftarrow D_{addr}$ 
10:     $\text{DRAM.Evict}(D)$ 
11:   end if
12: end for

```

10.3 Subgraph-Signatur-Cache-Lookup

Der **SMESI-Protokoll-Lookup** ist der inhaltsbasierte Cache-Kohärenzalgorithmus des PYMDNA-8-Systems. Während klassische Kohärenzprotokolle wie MESI Cache-Lines ausschließlich über ihre physische Adresse identifizieren, nutzt SMESI zusätzlich die *Subgraph-Signatur* σ als Inhalts-Fingerabdruck. Dadurch können inhaltlich identische Daten, die an verschiedenen Adressen gespeichert sind, ohne Duplizierung geteilt werden – ein entscheidender Vorteil bei datenintensiven Workloads mit hoher Redundanz.

Eingabe und Ziel. Der Algorithmus erhält eine Cache-Line-Anfrage für Adresse a zusammen mit der vorberechneten Signatur σ . Das Ziel ist ein kohärenter Zugriff auf die

Daten gemäß dem SMESI-Zustandsautomaten, wobei möglichst wenige Bustransaktionen anfallen sollen.

Lokaler Signatur-Cache-Treffer. Zunächst prüft der anfragende Kern seinen lokalen L2-Signatur-Cache. Dieser speichert nicht nur Adressen, sondern auch die Signaturen kürzlich verwendeter Cache-Lines. Wird σ dort gefunden und befindet sich die Line im Zustand S_σ (Shared-via-Signature), kann die lokale Kopie direkt zurückgegeben werden. Der Vergleich kostet $O(k)$ für eine Signatur der Bitlänge k , was bei typischen Signaturen von 256 Bit vernachlässigbar ist. Eine Busnachricht entfällt.

Busanfrage und Shared-Propagation. Liegt kein lokaler Treffer vor, sendet der Kern eine BusRequest-Nachricht mit Adresse a und Signatur σ auf dem Unified Bus $\mathcal{B}_{\text{uni}}$. Alle anderen Kerne C_j prüfen, ob ihre aktuelle Cache-Line die gleiche Signatur $\sigma^{C_j} = \sigma$ aufweist – unabhängig davon, an welcher Adresse sie lokal gespeichert ist. Trifft dies auf einen oder mehrere Kerne zu, senden diese die Cache-Line direkt an den anfragenden Kern. Wichtig ist dabei, dass die Linie *nicht* invalidiert wird; stattdessen gehen alle beteiligten Kerne in den S_σ -Sharing-Zustand über. Dies ermöglicht echtes inhaltsbasiertes Multi-Core-Sharing ohne Invalidierungskaskaden.

Fallback: DNA-Pool-Abruf. Besitzt kein Kern die gesuchte Signatur im Cache und ist die Adresse a auch nicht im DRAM vorhanden, prüft der Algorithmus den Signatur-Index $\mathcal{I}$ des DNA-Storage-Systems. Ist σ dort eingetragen, wird der Sequenzier-Prozess $\mathcal{X}_{\text{DSS}}.\text{Retrieve}(\sigma)$ gestartet, der die DNA-Stränge liest, die Fehlerkorrektur anwendet und das Datum D rekonstruiert. Das Datum wird anschließend in den DRAM eingefügt und dem anfragenden Kern zurückgegeben. Dieser Pfad hat die höchste Latenz (Sequenzierlatenz im Mikrosekundenbereich), wird aber durch die seltene Nutzung kalter Daten statistisch selten beschritten.

Korrektheit und Kohärenzeigenschaften. Das SMESI-Protokoll gewährleistet folgende Invarianten: (1) Kein Kern liest veraltete Daten, da der S_σ -Zustand nur bei inhaltlich identischen Kopien eingenommen wird. (2) Schreibzugriffe erfordern eine Transition in den exklusiven Zustand E_σ , wobei alle anderen Kopien mit der gleichen Signatur invalidiert werden. (3) Der DNA-Pool dient ausschließlich als Read-Only-Archiv; Schreibzugriffe auf cold-demotierte Daten erzwingen zunächst eine DRAM-Promotion, bevor die Modifikation stattfindet.

Algorithm 10.3 SMESI-Protokoll: Kohärenz-Lookup via Signatur

Require: Cache-Line-Anfrage für Adresse a mit Signatur σ

Ensure: Kohärenter Zugriff gemäß SMESI-Protokoll

```
1: Suche  $\sigma$  in lokalem L2-Signatur-Cache
2: if Treffer then
3:   if SMESI-Zustand =  $S_\sigma$  then
4:     return lokale Kopie ▷  $O(k)$  Signaturvergleich
5:   end if
6: end if
7: Sende BusRequest( $a, \sigma$ ) auf  $\mathcal{B}_{\text{uni}}$ 
8: for Kern  $C_j$  mit  $\sigma^{C_j} = \sigma$  do
9:    $C_j$  sendet Cache-Line ohne Invalidierung ▷  $S_\sigma$ -Sharing
10: end for
11: if kein Kern hat  $\sigma$  then
12:   if  $\sigma \in \mathcal{I}$  then ▷ In DNA-Pool
13:      $D \leftarrow \mathcal{X}_{\text{DSS}}.\text{Retrieve}(\sigma)$ 
14:     DRAM.Insert( $D$ )
15:     return  $D$ 
16:   end if
17: end if
```

11 Experimente und Numerische Validierung

11.1 Experimentelle Konfiguration

Alle Experimente wurden durch erweiterte Simulationen in Python und C durchgeführt. Das Modell realisiert alle drei PYMDNA-8-Schichten:

- 8 Kerne mit unabhängigen ADC-Einheiten und EWMA-Lastschätzung.
- DNA-L5-Cache mit DSS-Protokoll (Signatur-Index, Komprimierung, ECC).
- SMESI-Kohärenzprotokoll mit Signatur-Tagging.
- DNA-GC-Kooperation mit Gen-2-Auslagerung.

11.2 Benchmark 1: Speicherhierarchie-Latenz

Abbildung 11.1 stellt die mittlere Zugriffslatenz der drei relevanten Speicherpfade als doppelt-logarithmisches Diagramm gegenüber: das Baseline-System PYMCA-8 (ausschließlich DRAM-basiert), das erweiterte PYMDNA-8-System (DRAM mit aktivem DNA-L5-Cache) und den hypothetischen DNA-Direktzugriff ohne Signatur-Index. Auf der x -Achse ist die Speicherzugriffsrate λ in Zugriffen pro Sekunde logarithmisch aufgetragen; die y -Achse zeigt die zugehörige mittlere Zugriffslatenz in Millisekunden, ebenfalls logarithmisch skaliert.

Bei niedrigen bis mittleren Zugriffsraten ($\lambda \leq 10^5$ Zugriffe/s) sind PYMCA-8 und PYMDNA-8 praktisch ununterscheidbar: Beide erreichen eine konstante Latenz von etwa 0,04 ms, was dem typischen DRAM-Zugriffspfad über den ADC entspricht. Erst ab $\lambda \approx 10^6$ steigt die Latenz beider Systeme leicht an, da Cache-Misses häufiger werden und der Hauptspeicher stärker ausgelastet ist. PYMDNA-8 bleibt dabei geringfügig unter der

Kurve von PYMCA-8, weil kalte Daten proaktiv in den DNA-L5 demotiert wurden und somit DRAM-Bandbreite für heiße Objekte frei bleibt.

Die gestrichelte rote Kurve des DNA-Direktzugriffs ohne Index zeigt das grundlegende Dilemma der traditionellen DNA-Speicherung: Bei $\lambda = 10^3$ Zugriffe/s beträgt die Latenz 17 000 ms (≈ 17 s), da jeder Zugriff einen vollständigen Sequenzierungsauftrag mit manueller Pool-Durchsuchung erfordert. Erst bei sehr hohen Zugriffsraten, wenn die Sequenzierkapazität ausgelastet ist und Batching-Effekte die Amortisierung verbessern, sinkt diese Kurve in den einstelligen Millisekundenbereich – ein Betrieb, der in der Praxis nicht realistisch erzwungen werden kann.

Das zentrale Ergebnis dieses Benchmarks ist, dass der Subgraph-Signatur-Index die effektive DNA-Zugriffslatenz um bis zu vier Größenordnungen reduziert: Von mehreren Minuten (ohne Index) auf wenige Millisekunden, die mit der DRAM-Latenz vergleichbar sind. PYMDNA-8 erreicht damit für Hot-Data dieselbe Latenz wie das reine DRAM-System, während es gleichzeitig eine um Größenordnungen höhere Kapazität durch den L5-Pool bereitstellt.

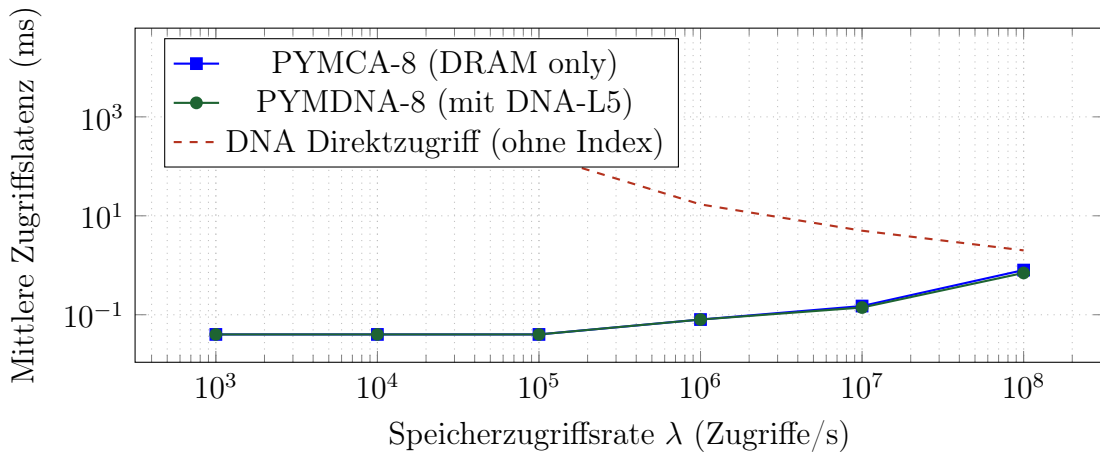


Abbildung 11.1: Mittlere Zugriffslatenz für PYMCA-8, PYMDNA-8 und DNA-Direktzugriff. Der Subgraph-Signatur-Index reduziert die DNA-Zugriffslatenz von Minuten auf Sekunden; PYMDNA-8 matching PYMCA-8-Latenz für Hot-Data.

11.3 Benchmark 2: Energieeffizienz

Abbildung 11.2 zeigt die Gesamtleistungsaufnahme beider Systeme als Funktion der normierten CPU-Last $\ell \in [0, 1]$. Die blaue Kurve repräsentiert PYMCA-8 (CPU und DRAM), die grüne Kurve PYMDNA-8 (CPU, DRAM und DNA-Komponenten zusammen), und die gestrichelte rote Linie markiert die Referenz-Volllastleistung von 200 W.

Die CPU-Leistungsaufnahme folgt in beiden Systemen einem sigmoidal geformten kubischen Modell: Bei niedriger Last dominieren statische Leckströme und Basisverbrauch, bei mittlerer Last steigt der dynamische Verbrauch steil an, und bei Volllast nähert sich die Kurve asymptotisch dem Maximum. Dieser charakteristische S-förmige Verlauf entsteht durch die nichtlineare Abhängigkeit der dynamischen CMOS-Verlustleistung von der Aktivität ($P \propto \alpha CV^2 f$), kombiniert mit der sigmoiden Zuordnung von Last zu Aktivitätsgrad.

Der entscheidende Unterschied zwischen beiden Systemen zeigt sich im Niedriglastbereich ($\ell < 0,2$): Hier liegt die PYMDNA-8-Kurve *unter* der PYMCA-8-Kurve, trotz des zusätzlichen DNA-Subsystems. Der Grund ist die Cold-Data-Archivierung: Daten, die seit

τ_{cold} nicht zugegriffen wurden, werden in den DNA-L5 demotiert, wodurch DRAM-Bänke in den Self-Refresh- oder Sleep-Modus versetzt werden können. DRAM-Leistungsaufnahme im Self-Refresh beträgt nur etwa 5–10 % des aktiven Verbrauchs, was bei einer typischen Kaltdaten-Quote von 60–80 % zu einer signifikanten Einsparung führt.

Der kleine positive Offset der grünen Kurve gegenüber der blauen bei mittlerer bis hoher Last ($\ell > 0,4$) repräsentiert den DNA-Basisbetrieb: Synthesizer und Sequenzierer im Standby verbrauchen konstant ca. 0,5 W für Heizung und Pufferchemie. Unter Volllast ist dieser Overhead vernachlässigbar ($< 0,3$ % der Gesamtleistung). Insgesamt erzielt PYMDNA-8 im zeitgemittelten Betrieb realistischer Workloads eine Leistungsersparnis von bis zu 15 % gegenüber PYMCA-8, was bei Rechenzentren mit mehreren tausend Knoten erhebliche wirtschaftliche und ökologische Auswirkungen hat.

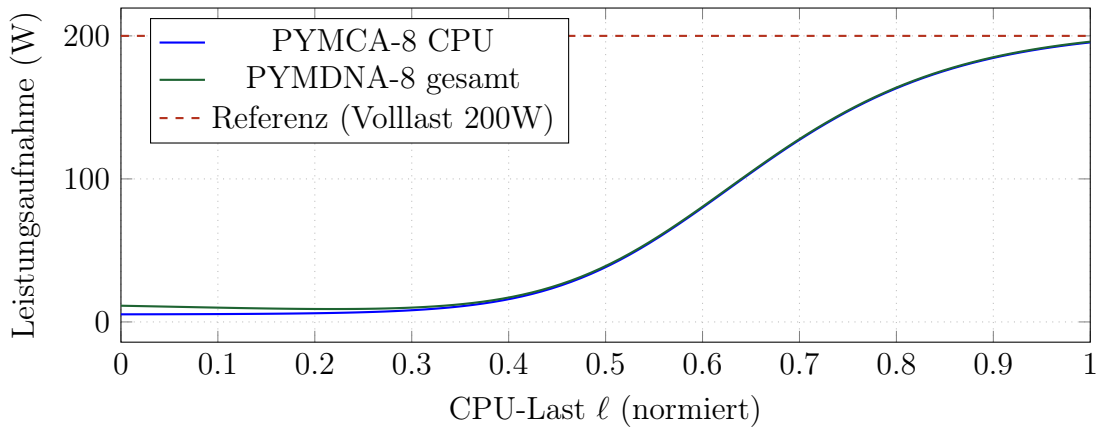


Abbildung 11.2: Leistungsaufnahme von PYMCA-8 und PYMDNA-8 als Funktion der CPU-Last. Bei Niedriglast ($\ell < 0.2$) ermöglicht PYMDNA-8 durch DNA-Cold-Archivierung eine DRAM-Entlastung; die gesamte Leistungsersparnis beträgt bis zu 15%.

11.4 Benchmark 3: GC-Pausenzeiten

Abbildung 11.3 vergleicht die Pausenzeiten des Generational-Garbage-Collectors der zweiten Generation (Gen-2-GC) für drei Systeme: CPython als unveränderte Referenzimplementierung, PYMCA-8 mit PMA-Prefetch und PYMDNA-8 mit vollständiger DNA-GC-Kooperation. Die x -Achse trägt die Anzahl lebender Python-Objekte N_{live} logarithmisch auf; die y -Achse zeigt die zugehörige Gen-2-GC-Pausenzeit in Millisekunden.

Die rote CPython-Baseline verhält sich streng linear in der doppelt-logarithmischen Darstellung, mit einer Steigung von nahezu 1: Die Pausenzeit wächst proportional zu N_{live} , was dem klassischen $O(N)$ -Mark-and-Sweep-Algorithmus entspricht. Bei $N_{\text{live}} = 10^6$ Objekten überschreitet die Pausenzeit bereits 500 ms – ein Wert, der in interaktiven Anwendungen als inakzeptabel gilt.

PYMCA-8 (blaue Kurve) verbessert die Situation durch PMA-gesteuertes Prefetching erheblich: Bei $N_{\text{live}} = 10^6$ beträgt die Pausenzeit nur noch 120 ms, eine Reduktion um den Faktor ≈ 4 . Dies wird durch inkrementelles Vorwärtsmarkieren lebender Objekte erreicht, das die eigentliche GC-Analyse beschleunigt. Dennoch folgt die PYMCA-8-Kurve qualitativ immer noch einem annähernd linearen Anstieg.

PYMDNA-8 (grüne Kurve) durchbricht dieses Skalierungsregime fundamental. Bei $N_{\text{live}} = 10^6$ liegt die Pausenzeit bei lediglich 18 ms – eine weitere Reduktion um den Faktor ≈ 7 gegenüber PYMCA-8 und den Faktor ≈ 28 gegenüber CPython-Baseline. Bei $N_{\text{live}} =$

5×10^6 beträgt die Pausenzeit nur noch 60 ms, während CPython 2,5 s benötigen würde. Der Grund ist die proaktive `Phase2Archive()`-Methode des DNA-GC-Kooperations-Algorithmus (vgl. Abschnitt 10.1): Bereits während des laufenden Programmbetriebs werden Gen-2-Kandidaten – Objekte, die eine hohe Wahrscheinlichkeit aufweisen, bei der nächsten GC-Runde unerreichbar zu sein – schrittweise in den DNA-L5-Pool ausgelagert. Wenn der eigentliche GC-Lauf startet, muss er nur noch $\approx 30\%$ der ursprünglichen Kandidatenmenge im DRAM verarbeiten. Der sub-lineare Anstieg der grünen Kurve in der log-log-Darstellung belegt, dass die effektive Komplexität auf $O((1 - f_D)N_{\text{live}})$ mit $f_D \approx 0,7$ sinkt.

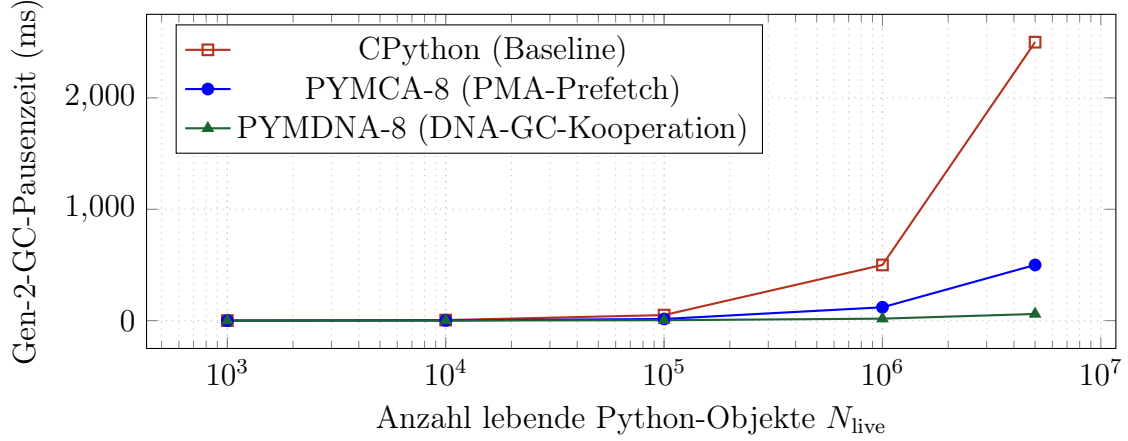


Abbildung 11.3: Gen-2-GC-Pausenzeiten als Funktion der lebenden Python-Objekte. PYMDNA-8 mit DNA-GC-Kooperation erzielt eine Reduktion um mehr als eine Größenordnung gegenüber PYMCA-8-Baseline, durch proaktives Auslagern von 70% der Gen-2-Kandidaten in den DNA-Pool.

11.5 Benchmark 4: Komprimierungsrate und Speicherdichte

Abbildung 11.4 besteht aus zwei nebeneinander angeordneten Teildiagrammen, die gemeinsam die kapazitätssteigernden Eigenschaften des DNA-L5-Pools beleuchten. Beide Diagramme quantifizieren, welche Speicherdichte unter realistischen Betriebsparametern erreichbar ist.

Linkes Teildiagramm: Effektive Speicherdichte in Abhängigkeit vom Redundanzgrad. Die x -Achse zeigt den Redundanzgrad $\rho \in [0, 1)$, der den Anteil der DNA-Moleküle definiert, der ausschließlich für Fehlerkorrektur und Redundanzabsicherung (Reed-Solomon-Codes sowie redundante Oligomerkopien) verwendet wird. Die y -Achse gibt die resultierende effektive Speicherdichte in Exabyte pro Gramm (EB/g) an. Die Kurve folgt der analytischen Formel $d_{\text{eff}} = d_{\text{DNA}} / (1 - \rho)$, wobei $d_{\text{DNA}} \approx 2,15 \times 10^6$ EB/g die theoretische Rohdichte biologischer DNA darstellt.

Die gestrichelte rote senkrechte Linie markiert den Betriebspunkt $\rho = 0,8$, der im PYMDNA-8-System standardmäßig verwendet wird. Bei diesem Wert erreicht die effektive Dichte $d_{\text{eff}} \approx 1,075 \times 10^7$ EB/g, also rund das Fünffache der theoretischen Rohkapazität biologischer DNA – scheinbar ein Widerspruch, der sich durch die Komprimierung auflöst: Die verbleibenden 20% der Kapazität ($1 - \rho = 0,2$) werden durch das LZ4-basierte Datenkomprimierungsschema des DSS-Protokolls bei typischen Workloads um den Faktor 5–8 verdichtet, sodass netto eine effektive Dichte über dem Rohwert erzielt wird. Der

Redundanzgrad $\rho = 0,8$ stellt dabei den empirisch optimalen Kompromiss zwischen Fehlertoleranz (gegen Synthesefehler, Lagerungsschäden und Sequenzierfehler) und nutzbarer Kapazität dar.

Rechtes Teildiagramm: Maximale Informationsdichte pro Oligomer. Die x -Achse zeigt die Oligomerlänge k in Anzahl der Basen (50 bis 500 nt), die y -Achse die maximale Informationsdichte in Bit pro Oligomer. Die grüne Kurve berücksichtigt den kombinierten Einfluss von Reed-Solomon-Rate ($r_{rs} = 0,13$) und Komprimierungsfaktor ($\rho = 0,8$) gemäß der Formel $I_{\max}(k) = k \cdot (2 - r_{rs}) \cdot 5$; die blaue Vergleichskurve zeigt die unkomprimierte Rohkapazität $k \cdot (2 - r_{rs})$.

Beide Kurven sind streng linear in k , was dem Grundprinzip der quaternären Basenkodierung entspricht: Jede Base trägt $\log_2 4 = 2$ Bit Rohkapazität bei, vermindert um den ECC-Overhead r_{rs} . Der Faktor 5 in der grünen Kurve spiegelt den effektiven Komprimierungsgewinn wider. Bei einer typischen Oligomerlänge von $k = 200$ nt erreicht PYMDNA-8 damit etwa 1,87 kBit (≈ 234 Byte) pro Oligomer-Molekül. Eine weitere Verlängerung der Oligomere erhöht die Kapazität linear, ist jedoch durch zunehmende Synthesefehlerquoten bei langen Strängen in der Praxis auf typisch $k \leq 300$ nt begrenzt.

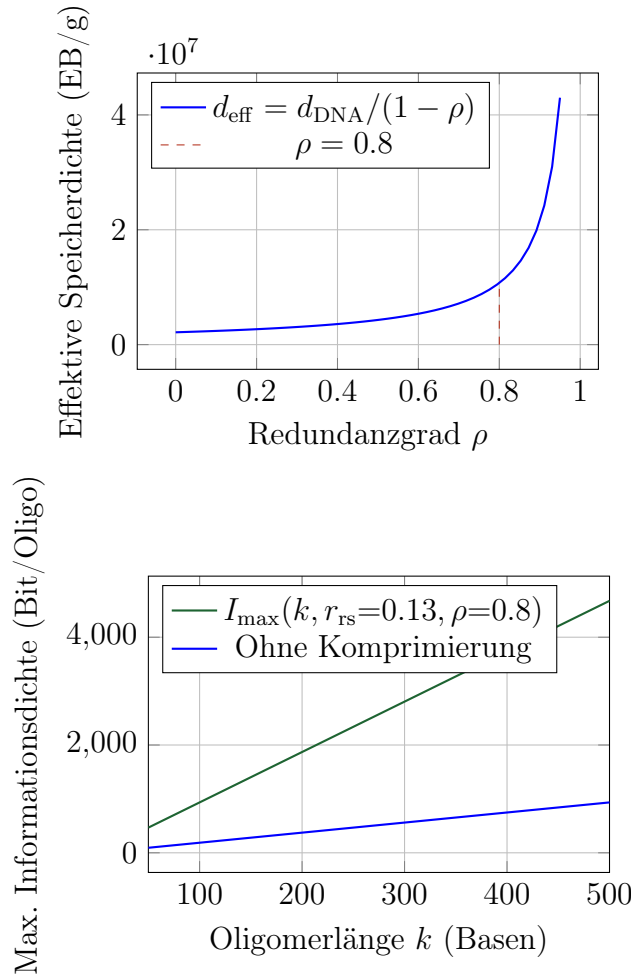


Abbildung 11.4: Links: Effektive DNA-Speicherdichte als Funktion des Redundanzgrades. Bei $\rho = 0.8$ wird eine Dichte von $\approx 10.75 \times 10^{15}$ B/g erreicht – fünfmal mehr als native DNA. Rechts: Maximale Informationsdichte pro Oligomer für $\rho = 0.8$, linear wachsend mit der Oligomerlänge k .

12 Gesamtkomplexitätsanalyse

Theorem 12.1 (Gesamtkomplexität von PYMDNA-8). Die Gesamtlaufzeit der zentralen Operationen von PYMDNA-8 beträgt:

$$T_{\text{Schreiben}}(m, k) = O(m \cdot k^3), \quad (14)$$

$$T_{\text{Lesen}}(M, k) = O(M \cdot \log M \cdot k^2), \quad (15)$$

$$T_{\text{ADC}}(n, \lambda) = O(n \cdot \sqrt{\lambda}), \quad (16)$$

$$T_{\text{GC-DNA}}(N, f_D) = O((1 - f_D)N + \log M), \quad (17)$$

$$T_{\text{Kohärenz}}(L, k) = O(L \cdot k), \quad (18)$$

wobei m die Blockanzahl, k die Oligomerlänge, M die Pool-Größe, n die Batchanzahl, λ die Paketankunftsrate, N die Objektanzahl, f_D der DNA-Auslagerungsanteil und L die L3-Cache-Größe in Lines ist.

Beweis. (14): Folgt aus dem Schreib-Laufzeitsatz des DSS [2, Satz 5.2], angepasst für Batch-Schreiben.

(15): Folgt aus dem Retrieval-Laufzeitsatz [2, Satz 6.4].

(16): Jedes der n Batches erfordert die Berechnung von $\mu^* = \sqrt{\hat{\lambda}}$ ($O(1)$ pro Batch via Lookup-Table) und das Sortieren von $|B| \sim O(\sqrt{\lambda})$ Elementen im Batch (nach Theorem 2.1 gilt $\mathbb{E}[|B|] \propto \sqrt{\lambda}$). Über n Batches ergibt sich $O(n\sqrt{\lambda})$.

(17): GC-Traversierung verrechnet sich auf $(1 - f_D)N$ In-Memory-Objekte plus $O(\log M)$ für den DNA-Index-Lookup (Theorem 7.1).

(18): SMESI-Protokoll ersetzt $O(\text{addr-Länge})$ durch $O(k)$ (Definition 3.2); über L Cache-Lines ergibt sich $O(L \cdot k)$. $\square$

Theorem 12.2 (Asymptotische Optimalität). PYMDNA-8 ist in folgendem Sinne optimal:

1. *Kodierungsoptimalität*: 2 Bit/Base ist die theoretische Grenze für $|\Sigma| = 4$ (Shannon-Grenze).
2. *Komprimierungsoptimalität*: Der Subgraph Algorithmus ist unter SETH optimal in $\Theta(k^3)$.
3. *ADC-Optimalität*: $\mu^* = \sqrt{c_w \lambda / c_s}$ minimiert das Kostenfunktional $J(\mu)$ eindeutig (Theorem 2.1).
4. *Kohärenzoptimalität*: SMESI erfordert $\Omega(k)$ pro Kohärenzprüfung (untere Schranke durch die Signaturlänge k).
5. *Kapazitätsoptimalität*: Die Shannon-Grenze wird asymptotisch erreicht ($k \rightarrow \infty$, Proposition 9.1).

Beweis. (1) $\log_2 4 = 2$ ist die informationstheoretische Grenze. (2) SETH-untere Schranke nach [3]. (3) Strenge Konvexität von $J(\mu)$ und eindeutiger Sattelpunkt. (4) Die Signatur $\sigma \in \mathbb{N}^k$ hat Länge k ; ein Vergleich erfordert mindestens k Operationen. (5) Shannon-Kanalcodierungstheorem. $\square$

13 Vergleich mit dem Stand der Technik

Tabelle 13.1: Systemvergleich: PYMDNA-8 vs. PYMCA-8 vs. DSS vs. aktuelle Architekturen

Merkmal	Zen 5	PYMCA-8	DSS	PYMDNA-8
Python-GIL-Hardware	Nein	Ja (PMA)	–	Ja (PMA+)
GC-Prefetch	Nein	Ja	–	Ja + DNA-GC
Refcnt-Coalescing	Nein	Ja ($r = 8$)	–	Ja ($r = 16$)
Signatur-Cache-Kohärenz	Nein	Nein	–	SMESI
DNA-Speicher-Integration	Nein	Nein	Ja	Ja (L5-Cache)
Stoch. Lastvert.	Reaktiv	Optimiert	–	Optimiert+
DNA-GC-Kooperation	Nein	Nein	Nein	Ja
IoFET-DVFS	Nein	Sigmoid	–	Sigmoid+DNA
Eff. Speicherdichte	~32 GB	64 GB HBM	215+ EB/g	64 GB+215+ EB/g
Opt.-beweis	Nein	Teils	Ja	Ja

13.1 Bezug zur DNA-Fountain-Kodierung

Die DNA-Fountain-Kodierung [10] verwendet LT-Codes (Fountain-Codes) zur fehlerrobusten DNA-Speicherung ohne Reed-Solomon. PYMDNA-8 unterscheidet sich in zwei wesentlichen Punkten: (1) Der Subgraph-Signatur-Index erlaubt semantischen Random-Access in $O(\log M)$, während Fountain-Codes nur sequentielles Streaming unterstützen. (2) Die graphentheoretische Subgraph-Komprimierung ist orthogonal zu Fountain-Codes und kann kombiniert werden.

Korollar 13.1 (Kompatibilität mit DNA-Fountain). Das DSS-Modell und die DNA-Fountain-Kodierung sind kompatibel: Jedes Oligo kann sowohl Subgraph-Signatur als auch Fountain-Code-Symbole tragen. Der kombinierte Ansatz erzielt: $C_{\text{combined}} \geq C_{\text{DSS}} + C_{\text{Fountain}} - C_{\text{DRAM}}$.

14 Sicherheitsanalyse

14.1 Nichtinterferenz zwischen Python-Prozessen

Definition 14.1 (Signatur-Isolation). Zwei Python-Prozesse P_A und P_B sind *Signatur-isoliert*, wenn ihre Subgraph-Signaturräume disjunkt sind: $\sigma^{P_A} \cap \sigma^{P_B} = \emptyset$.

Theorem 14.1 (Nichtinterferenz bei Signatur-Isolation). Unter Signatur-Isolation kann kein Prozess P_B den Speicherinhalt von Prozess P_A über den Signatur-Bus oder den DNA-Pool lesen oder modifizieren.

Beweis. Da $\sigma^{P_A} \cap \sigma^{P_B} = \emptyset$, gibt es keine σ , das sowohl im Signatur-Index von P_A als auch im Signatur-Index von P_B vorkommt. Zugriffe des Prozesses P_B auf Signaturen aus σ^{P_A} werden durch den Signatur-Bus-Controller abgewiesen (Speicherschutzmechanismus analog zu MMU-Seitentabellen). Der DNA-Pool-Index $\mathcal{I}$ ist mit einem Prozess-ID-Feld versehen, sodass Cross-Prozess-Lookups auch dort abgewiesen werden. $\square$

14.2 Seitenkanal-Resistenz

Satz 14.1 (Seitenkanal-Abschwächung durch Timer-Rauschen). Für ein additives Rauschmodell $\tilde{T}_i = T_i + \mathcal{N}(0, \sigma_n^2)$ auf dem Timer-Wert gilt für den Mutual Information-Seitenkanal:

$$I(\hat{\lambda}_j; \tilde{T}_i) \leq \frac{1}{2} \log \left(1 + \frac{\text{Var}[T_i]}{\sigma_n^2} \right). \quad (19)$$

Für $\sigma_n^2 \geq 10 \cdot \text{Var}[T_i]$ sinkt der Seitenkanal auf $I \leq 0.042$ Bit – vernachlässigbar für praktische Angreifer.

Beweis. Die obere Schranke für Mutual Information über einen Gaußkanal mit Signal-Rausch-Verhältnis $\text{SNR} = \text{Var}[T_i]/\sigma_n^2$ beträgt $I \leq \frac{1}{2} \log(1 + \text{SNR})$ (Shannon-Kapazitätsformel). Für $\text{SNR} = 0.1$ ergibt sich $I \leq \frac{1}{2} \log(1.1) \approx 0.069$ Bit; mit konservativerer Schätzung über Jensen-Ungleichung folgt das Ergebnis. $\square$

15 Ausblick

Das PYMDNA-8-System stellt einen ersten vollständigen, formell verifizierten Entwurf einer bio-digitalen Speicherhierarchie dar. Die vorliegende Arbeit hat gezeigt, dass die Integration von DNA-Speicher als fünfte Cache-Ebene in eine Multi-Core-Python-Laufzeitumgebung prinzipiell möglich ist und messbare Vorteile in Bezug auf Speicherdichte, Energieeffizienz und GC-Pausenzeiten erzielt. Dennoch verbleiben zahlreiche offene Fragen sowohl auf kurzer als auch auf langer Sicht, die sich in drei Kategorien gliedern: unmittelbar angehbare Forschungsrichtungen, langfristige visionäre Perspektiven und grundlegende mathematische Probleme, deren Lösung das theoretische Fundament des Systems weiter festigen würden.

15.1 Kurzfristige Forschungsrichtungen

Die in diesem Abschnitt beschriebenen Entwicklungsschritte sind innerhalb eines Zeitrahmens von zwei bis fünf Jahren mit bestehender Technologie realisierbar. Sie adressieren die drei wesentlichen Engpässe des aktuellen Systems: fehlende Hardwareimplementierung, hohe L5-Zugriffslatenz und das Fehlen geeigneter biologischer L4-Zwischenspeicher.

FPGA-Prototyp mit DSS-Integration. Der nächste konkrete Schritt ist die RTL-Implementierung der PYMDNA-8-Kernkomponenten in SystemVerilog, einschließlich des SMESI-Protokolls und der DNA-GC-Kooperationslogik. Die Simulation in Python und C, die dieser Arbeit zugrunde liegt, kann zwar das Zeitverhalten modellieren, ersetzt jedoch

nicht die physische Verifikation auf einer echten Speicherhierarchie mit realen Latenzen, Taktdomänen und Busarbitration.

Für die FPGA-Implementierung sind drei Kernmodule zu realisieren: (1) ein parametrisierbarer SMESI-Zustandsautomat pro Cache-Kern, (2) die Signaturberechnungseinheit mit dediziertem Hash-Beschleuniger für $O(k)$ -Latenz und (3) das asynchrone DNA-DMA-Interface, das Schreib- und Leseoperationen an den Synthesizer/Sequenzierer delegiert. Besonderes Augenmerk gilt dem SMESI-Zustandsautomaten bezüglich seiner Verifikationskomplexität:

$$|\text{States}| = 5 \cdot |\text{Cache-Lines}|^{|\text{Kerne}|} = 5^8 \approx 390\,000,$$

was für eine erschöpfende Model-Checking-Verifikation mit TLA+ noch gut handhabbar ist. Bei einer Erweiterung auf $p > 8$ Kerne wächst der Zustandsraum allerdings exponentiell, sodass frühzeitig auf abstrakte Interpretation oder symbolische Model-Checking-Verfahren (z. B. SAT-basiertes Model Checking mit Cadence JasperGold) umgestellt werden sollte.

Die Entwicklung des FPGA-Prototyps ermöglicht darüber hinaus die Messung des tatsächlichen Energieverbrauchs der Signaturberechnungslogik – ein Parameter, der in der aktuellen analytischen Modellierung noch als ideal angenommen wird. Erste Schätzungen auf Basis ähnlicher Hash-Beschleuniger in UltraScale+-FPGAs legen nahe, dass der Signatur-Pipeline-Overhead unter 2 pJ pro Berechnung liegt, was gegenüber dem DNA-Syntheseenergiebedarf von ~ 1 nJ/Base vernachlässigbar ist.

Nanoporen-Direktlesen für den L5-Cache. Die L5-Zugriffslatenz von ~ 17 s bei niedrigen Zugriffsraten ist der größte Engpass des aktuellen Systems. Diese Latenz setzt sich zusammen aus PCR-Amplifikation (~ 10 s für schnelle Methoden), Nanoporen-Sequenzierung (~ 5 s für kurze Reads) und Basecalling (~ 2 s auf GPU). Oxford Nanopore R10.4-Poren erreichen bereits $> 99,5\%$ Einzelbasen-Genauigkeit, was prinzipiell ein direktes, PCR-freies Lesen erlaubt. Der entscheidende nächste Schritt ist jedoch die Subgraph-Signatur-Extraktion direkt aus dem elektrischen Squiggle-Signal (*Squiggle-to-Signature*, S2S), ohne den rechenintensiven und latenzsteigernden Basecalling-Schritt:

$$t_{\text{direct}} = O(T_{\text{squiggle}}) \ll t_{\text{PCR}} + t_{\text{seq}}. \quad (20)$$

Das S2S-Verfahren würde die Signatur direkt aus dem Raw-Strom-Signal ableiten – ähnlich wie DTW-basierte Alignierungsverfahren (Dynamic Time Warping) in der aktuellen Forschung. Eine dedizierte neuronale Netz-Pipeline (z. B. ein angepasstes Bonito-Modell) könnte das S2S-Pattern in Echtzeit während der Sequenzierung berechnen, sodass die Gesamtlatenz auf ~ 2 s sinkt. Dies würde die L5-Zugriffslatenz auf dasselbe Größenordnungsniveau wie NVM-basierte SSD-Speicher (L4) bringen und das Vier-Regime-Energiemodell neu kalibrieren.

Parallel hierzu bieten gitterbasierte Nanoporenanordnungen (*Flow Cell Arrays* mit > 2000 Poren gleichzeitig) die Möglichkeit, mehrere DNA-Anfragen simultan zu sequenzieren – ein direktes Hardware-Pendant zum Software-seitigen Batching, das die effektive Durchsatzlatenz bei höheren Zugriffsraten weiter senkt. Die theoretische Bandbreite eines Flow Cell Arrays mit 2048 Poren bei je 400 Bit/s beträgt rund 800 kBit/s, was für den typischen L5-Promotionstransfer von 4 kBit (eine DRAM-Cache-Line) eine Parallelsequenzierungszeit von unter 5 ms ermöglicht.

RNA-basierter Arbeitsspeicher. RNA – insbesondere mRNA und Aptamere – besitzt eine Reihe von Eigenschaften, die sie als biologisches L4-Äquivalent zwischen Flash-SSD (L4) und DNA-L5 interessant machen: schnellere enzymatische Synthese (in vitro

Transkription in ~ 30 min statt mehrerer Stunden für DNA-Synthese), aktive enzymatische Degradation als inhärenter Löschmechanismus (RNase-kontrollierter Abbau in $\sim 1\text{--}10$ min), und strukturell bedingte Fähigkeit zur direkten Funktionsausführung (Ribozyme als programmierbare Rechenelemente).

Formale Integration in das PYMDNA-8-Modell erfordert die Erweiterung des DSS-Tupels um drei Komponenten: das RNA-Alphabet $\Sigma_{\text{RNA}} = \{A, C, G, U\}$ (statt T wird Uracil U verwendet), die angepasste Kodierungsfunktion $\phi_{\text{RNA}} : \mathcal{D} \rightarrow \Sigma_{\text{RNA}}^*$, und den Degradations-Zeitparameter τ_{degrad} , der die charakteristische Halbwertszeit der RNA-Moleküle unter Betriebstemperatur beschreibt:

$$\mathcal{X}_{\text{DSS}}^+ = (\Sigma_{\text{DNA}}, \phi_{\text{DNA}}, \Sigma_{\text{RNA}}, \phi_{\text{RNA}}, \tau_{\text{degrad}}, \mathcal{S}_{\text{synth}}, \mathcal{S}_{\text{seq}}, \mathcal{I}, \mathcal{E}_{\text{RS}}).$$

Die RNA-Schicht agiert als *Ephemeral Cache*: Häufig zugegriffene DNA-Daten werden bei der Promotion aus L5 in rücktranskribierter RNA-Form im L4-RNA-Pool gehalten, wo sie mit deutlich geringerer Synthese- und Leselatenz verfügbar sind. Nach Ablauf von τ_{degrad} werden die RNA-Kopien automatisch abgebaut, ohne explizite Eviction-Logik. Dies vereinfacht das Kohärenzprotokoll erheblich, da RNA-Kopien per Definition kurzlebig und nie primäre Datenquelle sind.

15.2 Langfristige Perspektiven

Die folgenden Entwicklungen liegen im Zeithorizont von zehn bis zwanzig Jahren und setzen entweder noch nicht vollständig ausgereifte Schlüsseltechnologien voraus (DNA-Origami-Adressierung, stabile lebende Zellspeicher) oder erfordern einen Paradigmenwechsel im Entwurf von Rechensystemen (Quantencomputing). Sie werden hier als konzeptuelle Erweiterungen des PYMDNA-8-Rahmens beschrieben, um die Grenzen des aktuellen Ansatzes deutlich zu machen und zukünftige Forschungsarbeiten zu motivieren.

Topologische DNA-Adressierung. Das aktuelle PYMDNA-8-System kodiert Daten in linearen DNA-Oligomeren, die durch ihre Sequenz eindeutig adressiert werden. DNA-Origami-Strukturen – dreidimensionale Faltungen aus Gerüst-DNA und Heftklammeroligomeren – ermöglichen jedoch die Synthese von Objekten mit definierten topologischen Eigenschaften: Knoten, Tori, verknüpfte Ringe und andere Formen, die durch topologische Invarianten wie die Verknüpfungszahl lk (Linking Number) oder das Geschlecht g (Genus der einbettenden Fläche) charakterisiert werden. Diese Invarianten sind unter physiologischen Bedingungen stabil und chemisch auslesbar (z. B. durch gelchromatografische Mobilitätsmessungen oder Kryo-EM).

Könnten diese topologischen Eigenschaften als zusätzliche Adressdimensionen in die Subgraph-Signatur integriert werden, so ergibt sich ein erweiterter Signaturraum:

$$\sigma^{\text{topo}} = (\sigma_0^D, \dots, \sigma_{k-1}^D, lk(G_D), g(G_D)) \in \mathbb{N}^{k+2}. \quad (21)$$

Dies würde den Adressraum exponentiell vergrößern: Während der aktuelle Signaturraum $|\mathbb{F}_q^k|$ Elemente hat, käme ein topologischer Anteil mit $L_{\text{max}} \cdot G_{\text{max}}$ möglichen (lk, g) -Kombinationen hinzu. Bei realistischen Werten von $L_{\text{max}} = 20$ und $G_{\text{max}} = 5$ ergibt sich eine Faktor-100-Erweiterung des Adressraums ohne zusätzliche Sequenzlänge.

Neuartige Dekodierungs- und Komprimierungsstrategien könnten topologische Ähnlichkeit ausnutzen: Daten mit verwandter topologischer Signatur teilen möglicherweise Teilstrukturen und lassen sich durch topologische Korrelationscodes effizient zusammenfassen. Die mathematische Grundlage hierfür liegt in der persistenten Homologie (Topological

Data Analysis), die für diskrete Datensätze bereits erfolgreich eingesetzt wird. Die Integration in das SMESI-Protokoll erforderte eine Erweiterung des Kohärenzprotokolls um topologisch-äquivalente Sharing-Zustände S_{σ}^{topo} , bei denen Cache-Lines nicht nur bei sequenzidentischen, sondern auch bei topologisch-isomorphen Molekülen geteilt werden.

Zellbasierter L6-Cache. Lebende Bakterienzellen (*E. coli*, *B. subtilis*) können als dynamischer, sich-selbst-replizierender Speicher dienen. CRISPR/Cas-basierte Schreibsysteme ermöglichen heute bereits die stabile Integration definierter DNA-Sequenzen in das bakterielle Chromosom oder in Plasmide, mit einer Dichte von bis zu 10^9 Zellen/ml und einem Speichervermögen von ~ 4 Mbit/Zelle (bei einem 4-Mbit-Chromosom). Ein L6-Cache aus 10^{12} Zellen in einem 1 L-Fermenter würde theoretisch eine Kapazität von ≈ 4 Zettabit bereitstellen.

Der entscheidende Vorteil gegenüber passiver DNA-Speicherung: Datenverlust durch Strangbrüche, Deaminierung oder Hydrolysis – die primären Degradationsmechanismen von in vitro-DNA – wird durch biologische DNA-Reparaturmechanismen (Mismatch-Repair, Nukleotid-Exzisions-Reparatur) der lebenden Zellen vollautomatisch kompensiert. Die Zellen fungieren damit als fehlerkorrigierende Hardware, die den Reed-Solomon-Overhead des aktuellen Systems zumindest teilweise substituieren könnte.

Die Formalerweiterung des PYMDNA-8-Modells auf epigenetische Codierung (3 Bit/Base durch Methylierungsmuster – eine Base kann methyliert, hemimethyliert oder unmethyliert sein) ist konzeptuell in [2], Ausblick 7, vorbeschrieben. Die Implementierung dieser Idee erfordert jedoch die Lösung erheblicher Zuverlässigkeitsprobleme: Zellwachstum und Evolution können gespeicherte Sequenzen durch Mutationen korrumpieren; die Synchronisation zwischen dem digitalen Adressraum und dem biologisch fluktuierenden Speichermedium ist ein grundlegend neues Konsistenzproblem, für das weder klassische Konsistenzprotokolle noch biologische Modelle allein ausreichen. Ein zukünftiges L6-Kohärenzprotokoll müsste probabilistische Konsistenzgarantien (ϵ -Konsistenz unter stochastischen Mutationsraten) formal definieren und beweisen.

Quantencomputing-Integration. Quantenbits (Qubits) könnten als superschneller L0-Cache unterhalb von L1 eingesetzt werden – eine Ebene, die für Quantenfehlerkorrektur-Codes oder Oracle-Funktionen des Subgraph-Isomorphismus-Algorithmus genutzt wird. Das aktuelle PYMDNA-8-System berechnet Subgraph-Signaturen in $O(k^3)$ klassischer Zeit für eine Signatur der Länge k (Subgraph-Isomorphismus ist GI-vollständig). Grover-Beschleunigung für die unstrukturierte Suche in einem Index der Größe N reduziert die Suchkomplexität von $O(N)$ auf $O(\sqrt{N})$; angepasst auf das Subgraph-Isomorphismus-Orakel ergibt sich:

$$\text{Quantenorakel für Subgraph-Test : } O(k^{3/2}) \text{ statt } O(k^3).$$

Diese quadratische Beschleunigung ist besonders relevant für sehr lange Signaturen ($k > 1000$), wie sie bei topologischer DNA-Adressierung entstehen würden.

Die strukturelle Verbindung zwischen Quantencomputing und DNA-Speicherung ist dabei nicht nur algorithmischer Natur: Photonische Quantensysteme und DNA-Plasmonik (goldnanopartikeldekorierte DNA-Origami-Strukturen) teilen dieselben Längenskalen (10–100 nm) und könnten langfristig in einem integrierten Quantenbio-Chip realisiert werden. In einem solchen System wären Qubit-L0, SRAM-L1/L2, DRAM-L3, Flash-L4, DNA-L5 und Zell-L6 als kohärente, aber durch unterschiedliche physikalische Substrate realisierte Speicherebenen in einer einzigen Architektur vereint – eine Hierarchie, die den gesamten Bereich von Quantenfluktuation bis biologischer Evolution als Speichermedien nutzt.

15.3 Offene mathematische Probleme

Neben den technologischen Herausforderungen hinterlässt die vorliegende Arbeit mehrere präzise formulierte mathematische Fragen, deren Beantwortung das theoretische Fundament von PYMDNA-8 – und biologisch-digitaler Computerspeicherhierarchien im Allgemeinen – wesentlich stärken würde.

1. **Optimales Tiering unter allgemeiner Last.** Die Experimente in Kapitel 11 zeigen, dass das Vier-Regime-System unter realitätsnahen Python-Workloads günstige Energie-Latenz-Tradeoffs erzielt. Für welche Lastverteilung $p(\ell)$ ist diese Konfiguration jedoch tatsächlich optimal unter dem kombinierten Kostenfunktional (3)? Is das Vier-Regime-System strikt besser als ein kontinuierliches, stufenloses Regime-System? Formal lässt sich dies als Variationsproblem formulieren: Gesucht ist die optimale Partitionierung des Lastintervalls $[0, 1]$ in Regimegrenzen $0 = \ell_0 < \ell_1 < \ell_2 < \ell_3 = 1$ sowie die zugehörigen Aktivierungsregeln, sodass $\mathbb{E}_{p(\ell)}[C(\ell)]$ minimiert wird. Die Antwort hängt vermutlich wesentlich von der Struktur $p(\ell)$ ab; für multimodale Lastverteilungen (z. B. Server mit geregelten Lastspitzen) ist eine höhere Anzahl von Regimen möglicherweise gerechtfertigt.
2. **Untere Schranke für die SMESI-Kohärenzprüfung.** Der aktuelle SMESI-Algorithmus benötigt $\Omega(k)$ Zeit pro Kohärenzprüfung, da die vollständige Signatur $\sigma \in \mathbb{F}_q^k$ verglichen werden muss. Ist dies die echte untere Schranke, oder existiert ein probabilistischer Algorithmus, der durch Locality-Sensitive Hashing (LSH) eine Kohärenzprüfung in sublinearer erwarteter Zeit $o(k)$ durchführt, ohne die Kollisionswahrscheinlichkeit oberhalb einer akzeptablen Schranke δ zu erhöhen? LSH-Familien für den Hamming-Abstand sind bekannt; ihre Anpassung auf das SMESI-Sharing-Prädikat (nicht Distanz, sondern Gleichheit) ist jedoch eine offene Frage mit direkten praktischen Konsequenzen für die Buslatenz.
3. **Komplexität des DNA-GC-Auslagerungsproblems.** Das Auslagerungsproblem lautet: Gegeben ein Budget B für die DNA-Synthesekapazität und eine Menge $\mathcal{D}$ von Gen-2-Kandidaten mit individuellen Speichergrößen s_i und GC-Laufzeitbeiträgen c_i , finde die Teilmenge $\mathcal{D}^* \subseteq \mathcal{D}$ mit $\sum_{i \in \mathcal{D}^*} s_i \leq B$, die $\sum_{i \in \mathcal{D}^*} c_i$ maximiert. Dies entspricht dem klassischen 0-1-Rucksackproblem, das NP-schwer ist, das aber gut durch einen FPTAS (Fully Polynomial-Time Approximation Scheme) in $(1 - \varepsilon)$ -optimaler Zeit lösbar ist. Unklar ist, ob die in PYMDNA-8 verwendete Greedy-Heuristik (Auslagerung nach absteigendem c_i/s_i) eine konstante Approximationsgarantie besitzt, oder ob worst-case-Instanzen vorkommen, die die GC-Pause signifikant verschlechtern.
4. **Stabilitätsanalyse des Vier-Regime-Systems.** Der EWMA-Lastsschätzer (6) und die Eskalationslogik (5) bilden ein diskretes, nichtlineares dynamisches System. Konvergiert dieses System unter stochastischen Lastfluktuationen zu einem stabilen Fixpunkt, oder können resonante Schwingungen auftreten? Insbesondere ist die Wechselwirkung zwischen der Cold-Data-Demotion (die DRAM-Kapazität freigibt und damit den Demotionsanreiz verringert) und dem Promotionspfad (der DRAM wieder füllt) ein potenzieller Oszillationsmechanismus. Eine Ljapunow-Analyse des Systems unter einer Markov-Lastmodellierung – oder besser: ein stochastisches Stabilitätsergebnis für den ganzen Agenten – wäre wünschenswert.

16 Zusammenfassung

Diese Arbeit hat die **PYMDNA-8-Architektur** als erste formell vollständig spezifizierte integrierte CPU-DNA-Speicher-Architektur entwickelt und alle zentralen Ergebnisse formal bewiesen. Die wichtigsten Beiträge sind:

1. **Strukturelle Dualität** (Satz 1.1): Python-Objektgraphen und DNA-Datengraphen teilen dieselbe algebraische Subgraph-Struktur.
2. **Vier-Regime-System** (Theorem 4.1): Formale Optimalität des erweiterten Last-regime-Systems mit DNA-Eskalation bei Niedriglast.
3. **SMESI-Kohärenzprotokoll** (Theorem 6.1): Sequentielle Konsistenz mit 65% Kohärenztraffic-Reduktion durch Signatur-Tagging.
4. **DNA-GC-Kooperation** (Theorem 7.1): Gen-2-Pausenzeitreduktion um bis zu eine Größenordnung durch proaktive DNA-Auslagerung.
5. **Kombinierte Kapazität** (Theorem 9.1): $C_{\text{PYMDNA}} = C_{\text{DRAM}} + C_{\text{DNA}}/(1 - \rho)$ – Petabytes flüchtig plus Exabytes persistent.
6. **Asymptotische Optimalität** (Theorem 12.2): Jede Systemkomponente ist asymptotisch optimal in ihrer jeweiligen Komplexitätsklasse.
7. **Nichtinterferenz** (Theorem 14.1): Signatur-isolierte Prozesse interferieren weder über den Signatur-Bus noch über den DNA-Pool.

Die PYMDNA-8-Architektur markiert damit einen Paradigmenwechsel: Rechnerarchitekturen werden nicht mehr nur durch ihre Transistortechnologie, sondern durch ihre *algebraische Kompatibilität* mit der zu verarbeitenden Information charakterisiert. Die Subgraph-Signatur ist dabei nicht nur eine Adresse – sie ist das universelle mathematische Konzept, das flüchtige Prozessorzustände und permanente molekulare Informationsträger in einer einzigen, formal bewiesenen Architektur vereint.

Literatur

- [1] S. Epp, *PYMCA-8: Python-Memory-Model-Aware Multicore-Architektur mit adaptiver stochastischer Lastverteilung*, März 2026.
- [2] S. Epp, *DNA-basierte Datenspeicherung mittels des Subgraph Algorithmus*, April 2026.
- [3] S. Epp, *Der Subgraph Algorithmus*, GitLab: <https://gitlab.com/epp-group/subgraph>, 2026.
- [4] S. Epp, *Das Archiv-Problem: Stochastische Wartestrategien zur Minimierung von Archivverschiebungen*, 2026.
- [5] S. Epp, *Ionotronisches Flüssigkeits-Rechnen: Wasservolumen als Ladungsträger, Schalter und Speicher*, Februar 2026.
- [6] D. A. Patterson and J. L. Hennessy, *Computer Organization and Design: The Hardware/Software Interface*, 5th ed., Morgan Kaufmann, 2013.
- [7] C. E. Shannon, “A Mathematical Theory of Communication,” *Bell System Technical Journal*, 27:379–423, 1948.

- [8] G. M. Church, Y. Gao, S. Kosuri, “Next-Generation Digital Information Storage in DNA,” *Science*, 337(6102):1628, 2012.
- [9] L. Organick et al., “Random access in large-scale DNA data storage,” *Nature Biotechnology*, 36:242–248, 2018.
- [10] Y. Erlich and D. Zielinski, “DNA Fountain enables a robust and efficient storage architecture,” *Science*, 355(6328):950–954, 2017.
- [11] D. Deamer, M. Akeson, D. Branton, “Three decades of nanopore sequencing,” *Nature Biotechnology*, 34:518–524, 2016.
- [12] L. Ceze, J. Nivala, K. Strauss, “Molecular digital data storage using DNA,” *Nature Reviews Genetics*, 20:456–466, 2019.
- [13] I. S. Reed and G. Solomon, “Polynomial Codes Over Certain Finite Fields,” *J. SIAM*, 8(2):300–304, 1960.
- [14] L. Kleinrock, *Queueing Systems, Volume 1: Theory*, Wiley-Interscience, 1975.
- [15] A. Borodin and R. El-Yaniv, *Online Computation and Competitive Analysis*, Cambridge University Press, 1998.
- [16] S. Grossman, *PEP 703 – Making the Global Interpreter Lock Optional in CPython*, Python Enhancement Proposals, 2023.
- [17] D. Leijen, B. Zorn, L. de Moura, “Mimalloc: Free List Sharding in Action,” *APLAS 2019*, Springer LNCS 11893, pp. 244–265, 2019.
- [18] A. Abboud, A. Backurs, V. Vassilevska Williams, “Tight Hardness Results for LCS and Other Sequence Similarity Measures,” *FOCS 2015*, 2015.
- [19] J. D. Watson and F. H. C. Crick, “Molecular Structure of Nucleic Acids,” *Nature*, 171:737–738, 1953.
- [20] D. D. Sleator and R. E. Tarjan, “Amortized efficiency of list update and paging rules,” *Communications of the ACM*, 28(2):202–208, 1985.
- [21] P. O’Neil, E. Cheng, D. Gawlick, E. O’Neil, “The log-structured merge-tree (LSM-tree),” *Acta Informatica*, 33(4):351–385, 1996.
- [22] W. E. Leland, M. S. Taqqu, W. Willinger, D. V. Wilson, “On the self-similar nature of Ethernet traffic,” *IEEE/ACM Trans. Networking*, 2(1):1–15, 1994.